





INSTITUT POLYTECHNIQUE DE PARIS

Master 2 Quantum, Mathematics, and Computer Science (QMI)

Internship Report

Identifying Decoding Failures in Quantum Error Correction

Author:
Leqi LIU

Degree:
Master in Computer Science

Supervisor:
Prof. Nicolas Sangouard
Co-supervisors:
Dr. Kiara Hansenne and MSc. Anthony Benois
Institut de Physique Théorique, CEA Saclay

Academic referent:
Prof. Marco Fanizza
Institut Polytechnique de Paris

September 2026

Institut de Physique Théorique, CEA Saclay
Université Paris-Saclay, CNRS, CEA
91191 Gif-sur-Yvette, France

Abstract

Belief propagation with ordered-statistics decoding (BP+OSD) is a practical decoder for quantum low-density parity-check codes, but its performance can be strongly affected by message dynamics and the choice of Tanner graph. In this work, we investigate a low-noise discrepancy between BP+OSD and the search-based Tesseract decoder on triangular codes under circuit-level noise.

By progressively simplifying the noise model and analyzing individual low-weight failures, we separate three aspects of BP decoding: message dynamics, syndrome resolution, and logical class discrimination. Message instability is strongly associated with failure to reach a syndrome-consistent state, but is only weakly informative about logical correctness once the syndrome constraints are satisfied. Moreover, an incorrect BP hard decision can retain useful soft information for OSD, and more informative reliability orderings may occur transiently before the final BP iteration.

Interventions further reveal a strong dependence on the parity-check representation. Equivalent parity-check matrices repair substantially more disagreement cases than additional iterations or repeated runs on the original graph. An Adaptive BP intervention based on redundant checks confirms this effect, while equal-budget ablations show that most of the gain arises from representation diversity rather than the instability-based selection heuristic itself.

These results suggest that BP failures in the studied circuit-level setting cannot be characterized by instability alone: syndrome resolution, logical ambiguity, and Tanner-graph representation must be treated as distinct components of the decoding problem.

Contents

1 Introduction **1**

2 Quantum Error Correction **3**
2.1 Stabilizer Formalism . . . 3
2.1.1 Binary Symplectic Representation . . . 6
2.2 Operator Basis and TLS Decomposition . . . 6
2.3 Decoding Problem . . . 8
2.4 Tanner Graph . . . 9
2.5 Noise Models . . . 9
2.5.1 Detector Error Model . . . 10
2.6 Waterfall and Error-floor Region . . . 11
2.6.1 Logical Error Rate and Threshold Behaviour . . . 11
2.6.2 Dominant Low-Weight Failures . . . 12

3 Quantum Error Correcting Codes **15**
3.1 CSS Code . . . 15
3.2 Quantum LDPC Code . . . 16
3.2.1 Triangular Code . . . 16

4 Decoders **18**
4.1 BP Decoder . . . 18
4.2 BP+OSD Decoder . . . 19
4.3 Complexity Analysis . . . 20

5 BP Failure Mechanisms **21**
5.1 Cycles . . . 21
5.2 Trapping Sets . . . 21
5.2.1 Classical-type Trapping Sets . . . 22
5.2.2 Symmetric Stabilizer Trapping Sets . . . 22
5.3 Oscillating Region . . . 22

6 Experiment **23**
6.1 Experiment Methodology . . . 23
6.1.1 Simulation Settings . . . 23
6.1.2 Experimental Sequence . . . 23
6.2 Phenomenon Observation . . . 24
6.2.1 Isolating the BP+OSD and Tesseract Discrepancy . . . 24
6.2.2 Controlled Decoder Diagnostics: CNOT Schedule, BP Schedule and Forced OSD 24
6.3 Failure Cases Overlap and Low-weight Errors . . . 26
6.4 Message Dynamics: What Instability Measures and What It Does Not . . . 26
6.4.1 Min-sum Scaling and the Limits of Oscillation Suppression . . . 29
6.4.2 Early Stopping: a Real but not Universal Mechanism . . . 30
6.5 Taxonomy of BP Failures . . . 32
6.6 From BP to OSD . . . 32
6.6.1 BP Beliefs and OSD Reliability . . . 32
6.6.2 Reliability Windows and Online Selection . . . 34
6.7 Mechanism Attribution and Parity-check Matrix Dependence . . . 35
6.8 Representation-Based Interventions . . . 36
6.8.1 Adaptive BP . . . 36
6.8.2 Adaptive BP Repair is Strongly Mechanism Dependent . . . 36
6.8.3 Ablation: Parity-check Matrix Diversity . . . 38

6.9 Experiment Summary 38

7 Conclusion **40**

A Supplementary Material **41**

Acknowledgements

I would like to thank Prof. Nicolas Sangouard for supervising this internship. I am grateful for the opportunity to work on this project and for the freedom to explore different directions throughout the internship.

I am especially grateful to Kiara Hansenne and Anthony Benois for their close involvement in this work. Kiara provided many valuable suggestions throughout the project and was particularly helpful in reviewing and improving the report. Anthony offered extensive guidance during the research, helping me think through experimental designs.

I would also like to thank the members of the QIT group for the readings, discussions and welcoming research environment.

Finally, I would like to thank my friends and family for their support throughout the year.

1 Introduction

Quantum error correction is essential for fault-tolerant quantum computation. A quantum error-correcting code encodes logical information into a larger physical system and repeatedly measures stabilizer checks. The measurement outcomes are summarized by a *syndrome*, a binary vector indicating which stabilizer checks are triggered. The decoding problem is then to infer a correction that is both syndrome-consistent and logically equivalent to the physical error. For a family of codes called quantum low-density parity-check (qLDPC) codes, the parity constraints can be represented by a *Tanner graph*, a bipartite graph connecting variable nodes representing possible errors to check nodes representing syndrome constraints. Belief propagation (BP) is an attractive decoding method because it exploits the sparsity of this graph and has approximately linear complexity per iteration [24]. However, BP is exact only on cycle-free Tanner graphs. The Tanner graphs of practical quantum codes generally contain many short cycles and highly correlated local structures, often referred to as trapping sets [26], which may lead to oscillating messages, overconfident beliefs, or convergence towards an incorrect fixed point. In addition, the degeneracy of quantum codes means that different physical error patterns may correspond to the same logical correction class, further complicating the relation between local marginal estimates and successful quantum decoding [30].

A widely used approach is therefore to combine BP with a post-processing step such as ordered-statistics decoding (OSD) [23]. OSD uses the posterior reliabilities produced by BP to construct a small list of syndrome-consistent candidates and has demonstrated strong performance across several qLDPC code families [23, 30]. However, OSD introduces Gaussian-elimination and candidate-enumeration costs, while its effectiveness depends on the reliability information inherited from BP. Recent work has explored several alternatives: BP with guided decimation sequentially fixes reliable variables to encourage convergence without solving a linear system [35], while BP with ordered Tanner forest removes parts of the decoding graph until a tree-like subproblem is obtained [14]. Reliable subset reduction and approximate degenerate OSD reduce the size of the post-processing problem by fixing highly reliable variables and explicitly removing some degenerate candidates [19]. At the graph level, graph augmentation and rewiring for inference (GARI) modifies the Tanner graph to eliminate harmful short cycles while preserving the underlying decoding problem [20]. Very recent multiple-bases BP methods similarly construct redundant parity-check representations and run BP over several modified Tanner graphs [25].

Search-based methods provide a complementary approach. In particular, the Tesseract decoder searches over error subsets to find a low-cost error configuration consistent with the observed syndrome [3]. Recent variants of OSD have also reconsidered the conventional BP+OSD stopping rule. Best-First OSD, for example, invokes OSD after a fixed small number of BP iterations rather than waiting for BP failure, motivated by the unreliability of BP convergence under full circuit-level noise [2]. These developments suggest that BP convergence, syndrome consistency, and decoding success should be treated as distinct properties.

In this work, we use the discrepancy between BP+OSD and the Tesseract decoder as a controlled setting for studying BP+OSD failure mechanisms under circuit-level noise. We focus primarily on small triangular codes, where low-weight error configurations can be enumerated and the complete evolution of BP messages, beliefs, syndrome consistency, and logical outcomes can be tracked directly. This setting allows decoder failures to be studied at the level of individual trajectories rather than only through aggregate logical error rates.

Our analysis separates the decoding process into three layers: BP message dynamics, resolution of the syndrome constraints, and discrimination between logical classes. We show that these layers need not coincide. BP may remain dynamically unstable without satisfying the syndrome, may reach a syndrome-consistent but logically wrong state, or may converge numerically to such a state. Conversely, even when the BP hard decision is wrong, its posterior beliefs can still contain sufficient information for OSD to recover the correct logical class.

We further observe that BP depends on the Tanner graph representation. By applying equivalent parity-check transformations to the same decoding problem, we find that many BP failures can be repaired without changing either the syndrome or the priors. This motivates an Adaptive BP in-

tervention based on redundant parity checks. Equal-budget ablations further separate the effect of additional computation from that of graph diversity, showing that the latter accounts for most of the observed improvement in the tested setting.

Together, these experiments support a mechanism-based view of BP decoding: message dynamics primarily govern syndrome resolution, while logical class selection constitutes a distinct problem that cannot in general be diagnosed or repaired by convergence criteria alone.

This report is organized as follows: Chapter 2 introduces the foundations of quantum error correction, including the stabilizer formalism, decoding problem, Tanner graph, and the waterfall and error-floor regions. Chapter 3 presents the code families considered in this work, with particular emphasis on the triangular code used in the numerical study. Chapter 4 describes the BP and BP+OSD decoders together with their main algorithms and computational costs. Chapter 5 outlines the structural and dynamical mechanisms associated with BP failures. Chapter 6 details the simulation methodology and numerical experiments. Finally, Chapter 7 summarizes the main findings, discusses their limitations, and outlines directions for future work.

2 Quantum Error Correction

Quantum error correction protects encoded information by inferring errors from syndrome measurements without directly measuring the logical state [13]. Because many data and circuit faults can be compatible with the same observed syndrome, decoding requires a precise representation of the code constraints, error configurations, and noise process. This chapter therefore introduces stabilizer codes and their binary symplectic representation, Tanner graphs for sparse decoding constraints, and detector error models for noisy syndrome-extraction circuits.

2.1 Stabilizer Formalism

Quantum error correction protects logical quantum information by encoding it redundantly into physical qubits, such that errors can be diagnosed without directly measuring the encoded logical state [13]. In this work, we restrict attention to stabilizer codes, where measurements of commuting stabilizer operators produce an error syndrome that is used by a classical decoder to infer a Pauli correction. Rather than being physically applied, this correction may be tracked classically in a *Pauli frame* [29]. Assuming a stochastic Pauli noise model [34], and using the fact that Clifford circuits propagate Pauli errors to Pauli errors, the decoding problem can then be formulated as probabilistic inference over discrete Pauli error configurations [13].

Definition 2.1.1 (Pauli group)**.** The n-qubit Pauli group is defined as

$$\mathcal{P}_n := \{\alpha P_1 \otimes \cdots \otimes P_n \mid \alpha \in \{\pm 1, \pm i\},\ P_i \in \{I, X, Y, Z\}\}\,, \tag{2.1}$$

where I is the identity operator and X, Y, Z are the single-qubit Pauli operators. They satisfy $X^2 = Y^2 = Z^2 = I$ and pairwise anticommute.

Although phases are retained in the definition of $\mathcal{P}_n$, global phases are physically irrelevant. We therefore identify Pauli operators that differ only by an overall phase and define the projective Pauli group

$$\overline{\mathcal{P}}_n := \mathcal{P}_n / \{\pm I, \pm iI\}. \tag{2.2}$$

Definition 2.1.2 (Stabilizer code)**.** An $[\![n, k, d]\!]$ stabilizer code $\mathcal{C}$ that encodes k logical qubits into n physical qubits is specified by an Abelian subgroup $\mathcal{S} \subset \mathcal{P}_n$, called the *stabilizer group*, such that $-I \notin \mathcal{S}$. It can be generated by $n-k$ independent commuting Pauli operators, called the *stabilizer generators*,

$$\mathcal{S} = \langle S_1, \ldots, S_{n-k} \rangle, \tag{2.3}$$

and therefore contains 2^{n-k} elements. The codespace is simultaneous $+1$ eigenspace of all stabilizers,

$$\mathcal{H}_{\mathcal{C}} := \left\{ |\psi\rangle \in (\mathbb{C}^2)^{\otimes n} : S\,|\psi\rangle = |\psi\rangle\,,\ \forall S \in \mathcal{S} \right\}. \tag{2.4}$$

Measuring a generator S_i yields an outcome ± 1, with corresponding projectors

$$\Pi_i^{(\pm)} := \frac{I \pm S_i}{2}. \tag{2.5}$$

Consequently, the projector onto the codespace, corresponding to obtaining the $+1$ outcome for every generator, is

$$\Pi_{\mathcal{C}} := \prod_{i=1}^{n-k} \frac{I + S_i}{2} = \frac{1}{|\mathcal{S}|} \sum_{S \in \mathcal{S}} S. \tag{2.6}$$

The stabilizer group describes operations that act trivially on encoded states. To identify Pauli operations that preserve the codespace but act non-trivially within it, we consider its normalizer.

Definition 2.1.3 (Normalizer and logical Pauli group)**.** The normalizer of the stabilizer group in the

Pauli group is

$$\mathcal{N}(\mathcal{S}) := \left\{ P \in \mathcal{P}_n : P\mathcal{S}P^\dagger = \mathcal{S} \right\}. \tag{2.7}$$

The centralizer of the stabilizer group in the Pauli group is

$$\mathcal{C}_{\mathcal{P}_n}(\mathcal{S}) := \{ P \in \mathcal{P}_n : PS = SP, \quad \forall S \in \mathcal{S} \}. \tag{2.8}$$

For a stabilizer group, the normalizer in $\mathcal{P}_n$ coincides with its centralizer,

$$\mathcal{N}(\mathcal{S}) = \mathcal{C}_{\mathcal{P}_n}(\mathcal{S}), \tag{2.9}$$

since Pauli operators either commute or anticommute and $-I \notin \mathcal{S}$.

Logical Pauli operators are represented by equivalence classes modulo stabilizers, giving

$$\mathcal{L} := \mathcal{N}(\mathcal{S})/\mathcal{S}. \tag{2.10}$$

Two elements in the same coset differ by a stabilizer and therefore induce the same action on the codespace.

Errors outside the normalizer do not preserve the codespace. Instead, their commutation relations with the stabilizer generators determine which syndrome subspace the corrupted state occupies.

Definition 2.1.4 (Syndrome)**.** Let $E \in \mathcal{P}_n$ be a Pauli error and $\{S_1, \ldots, S_{n-k}\}$ a chosen set of stabilizer generators. For an encoded state $|\psi\rangle$, the corrupted state $E|\psi\rangle$ is an eigenstate of each S_i,

$$S_i E |\psi\rangle = (-1)^{s_i} E |\psi\rangle, \tag{2.11}$$

where

$$s_i = \begin{cases} 0, & ES_i = S_i E, \\ 1, & ES_i = -S_i E. \end{cases} \tag{2.12}$$

The binary vector

$$s = s(E) = \left(s_1, \ldots, s_{n-k}\right) \in \mathbb{F}_2^{n-k} \tag{2.13}$$

is called the *syndrome* of E. Thus, s_i encodes the outcome $(-1)^{s_i}$ of measuring the stabilizer generator S_i. The joint projector onto the subspace associated with a syndrome s is

$$\Pi_s := \prod_{i=1}^{n-k} \frac{I + (-1)^{s_i} S_i}{2}. \tag{2.14}$$

In particular, the trivial syndrome $s = 0$ corresponds to the codespace, so that $\Pi_0 = \Pi_{\mathcal{C}}$.

Equivalently, the syndrome records which stabilizer generators anticommute with the error. It therefore provides information about the error without directly measuring the encoded logical state. In general, however, it does not identify a unique physical error because distinct errors may share the same syndrome.

Proposition 2.1.5 (Classification of errors)**.** *A Pauli error E can be classified as:*

1. *if $E \in \mathcal{S}$, then E acts trivially on the codespace;*
2. *if $E \in \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$, then E preserves the codespace but acts as a non-trivial logical Pauli operator;*
3. *if $E \notin \mathcal{N}(\mathcal{S})$, then E moves the state to a non-trivial syndrome subspace and is detectable by at least one stabilizer measurement.*

Among the undetectable errors, the smallest non-trivial logical operator determines the code's distance and hence its worst-case correction guarantee.

Definition 2.1.6 (Code distance)**.** The distance of a stabilizer code is the minimum weight of a non-trivial logical Pauli operator

$$d := \min_{P \in \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}} \mathrm{wt}(P), \tag{2.15}$$

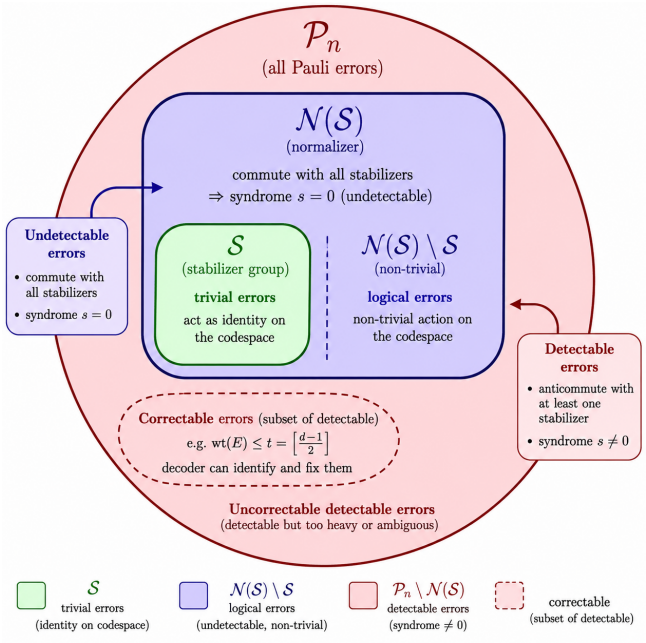


Figure 2.1: Classification of Pauli errors with respect to the stabilizer group $\mathcal{S}$ and its normalizer $\mathcal{N}(\mathcal{S})$. Errors in $\mathcal{S}$ act trivially on the codespace, while errors in $\mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$ are undetectable non-trivial logical operators. Errors outside $\mathcal{N}(\mathcal{S})$ anticommute with at least one stabilizer and therefore produce a non-zero syndrome. Among these detectable errors, only a subset is guaranteed correctable, i.e., all errors of weight at most $\lfloor (d-1)/2 \rfloor$ for a code of distance d.

where $\mathrm{wt}(P)$ is the number of qubits on which P acts non-trivially, called the *weight* of the Pauli operator.

Corollary 2.1.7 (Error detection and correction)**.** *An $[\![n, k, d]\!]$ stabilizer code detects every Pauli error of weight smaller than d and corrects every Pauli error of weight at most $t = \left\lfloor \frac{d-1}{2} \right\rfloor$.*

Distance provides a worst-case guarantee based only on error weight. More generally, correctability is characterized by the Knill–Laflamme condition [18]. A set of errors $\{E_a\}$ is correctable if and only if

$$\Pi_{\mathcal{C}} E_a^\dagger E_b \Pi_{\mathcal{C}} = c_{ab} \Pi_{\mathcal{C}}, \qquad \forall\, a, b, \tag{2.16}$$

for some coefficients c_{ab}, where $\Pi_{\mathcal{C}}$ is the projector onto the codespace, defined in Eq. (2.6).

Proposition 2.1.8 (Pauli error-correction condition)**.** *A collection of Pauli errors $\{E_a\}$ is correctable if, for every pair E_a, E_b, either $E_a^\dagger E_b \notin \mathcal{N}(\mathcal{S})$, so that the two errors have different syndromes, or $E_a^\dagger E_b \in \mathcal{S}$, so that the two errors act identically on the codespace [13, 18].*

Definition 2.1.9 (Degeneracy)**.** Two Pauli errors E_a and E_b are said to be *degenerate* with respect to a stabilizer code if

$$E_a^\dagger E_b \in \mathcal{S}. \tag{2.17}$$

They therefore have the same syndrome and induce the same action on the codespace. More generally, a code is said to be degenerate with respect to an error set which contains distinct degenerate errors [13].

Degeneracy implies that a decoder need not identify the physical error uniquely: errors that differ by a stabilizer are equivalent for the purpose of recovery. In contrast, the remaining case,

$$E_a^\dagger E_b \in \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}, \tag{2.18}$$

represents a harmful logical ambiguity. The two errors have the same syndrome but differ by a non-trivial logical Pauli operator. Therefore, the syndrome alone cannot determine the logical coset of the error.

2.1.1 Binary Symplectic Representation

Definition 2.1.10 (Binary representation of Pauli operators)**.** Ignoring global phase, every n-qubit Pauli operator $P \in \mathcal{P}_n$ can be represented by a binary vector [13]:

$$v(P) := \begin{pmatrix} x \\ z \end{pmatrix} \in \mathbb{F}_2^{2n}, \quad x = (x_1, \ldots, x_n)^T,\ z = (z_1, \ldots, z_n)^T \in \mathbb{F}_2^n \tag{2.19}$$

through the correspondence $P \sim X^{x_1} Z^{z_1} \otimes \cdots \otimes X^{x_n} Z^{z_n}$. On each qubit,

$$I \leftrightarrow \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \qquad X \leftrightarrow \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad Z \leftrightarrow \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \qquad Y \leftrightarrow \begin{pmatrix} 1 \\ 1 \end{pmatrix}. \tag{2.20}$$

Multiplication of Pauli operators corresponds to binary addition. In particular, the binary representation of a Pauli error E is denoted by $e = v(E)$ while the binary representation of a stabilizer generator S_i is denoted by $g_i = v(S_i)$. Their commutation relations are encoded by the symplectic inner product.

Definition 2.1.11 (Symplectic inner product)**.** For $v = \begin{pmatrix} x \\ z \end{pmatrix}$ and $v' = \begin{pmatrix} x' \\ z' \end{pmatrix}$, define

$$\langle v | v' \rangle_{\mathrm{s}} := x^T z' + z^T x'. \tag{2.21}$$

Proposition 2.1.12 (Commutation relation)**.** *Two Pauli operators represented by v and v' commute if and only if $\langle v|v'\rangle_{\mathrm{s}} = 0$, and anticommute if and only if $\langle v|v'\rangle_{\mathrm{s}} = 1$.*

Proof. If $P(v)$ and $P(v')$ are the Pauli operators represented by v and v', then using $ZX = -XZ$ on each qubit gives

$$P(v)P(v') = (-1)^{x^T z' + z^T x'} P(v')P(v) = (-1)^{\langle v|v'\rangle_{\mathrm{s}}} P(v')P(v). \tag{2.22}$$

Hence they commute when $\langle v|v'\rangle_{\mathrm{s}} = 0$ and anticommute when $\langle v|v'\rangle_{\mathrm{s}} = 1$. □

Let

$$G_{\mathcal{S}} := \begin{bmatrix} g_1^T \\ \vdots \\ g_{n-k}^T \end{bmatrix} = \begin{bmatrix} G_X \mid G_Z \end{bmatrix} \in \mathbb{F}_2^{(n-k)\times 2n}. \tag{2.23}$$

be the stabilizer generator matrix whose rows are the binary representations of the chosen stabilizer generators, and define

$$\Lambda := \begin{bmatrix} 0 & I_n \\ I_n & 0 \end{bmatrix}. \tag{2.24}$$

Proposition 2.1.13 (Stabilizer commutation and syndrome equations)**.** *The stabilizer generators commute pairwise if and only if*

$$G_{\mathcal{S}} \Lambda G_{\mathcal{S}}^T = 0. \tag{2.25}$$

For a Pauli error represented by e, the syndrome is given by

$$s = G_{\mathcal{S}} \Lambda e. \tag{2.26}$$

Defining the parity-check matrix $H = G_{\mathcal{S}}\Lambda$, Eq. (2.26) becomes $He = s$. Moreover, Eq. (2.25) becomes $HG_{\mathcal{S}}^T = 0$

2.2 Operator Basis and TLS Decomposition

The stabilizer and logical operators do not by themselves form a complete basis of the n-qubit Pauli group. Additional operators are required to represent the syndrome degrees of freedom. Extending the stabilizer generators to a canonical symplectic basis leads to the TLS decomposition [16].

Definition 2.2.1 (Pure-error generators)**.** A set of pure-error generators, also called destabilizers, is a collection $\{T_i\}_{i=1,\dots,n-k}$ such that

$$T_i S_j = (-1)^{\delta_{ij}} S_j T_i. \tag{2.27}$$

That is, T_i anticommutes with S_i and commutes with every S_j for $j \neq i$. The generators may be chosen to commute with one another and with a selected set of logical Pauli representatives.

The name *pure error* does not mean that $T(s)$ is the physical or most likely error. It is a convention-dependent representative chosen only to reproduce a prescribed syndrome.

Definition 2.2.2 (TLS decomposition)**.** Choose stabilizer generators $\{S_i\}$, pure-error generators $\{T_i\}$, and logical representatives $\{\overline{X}_j, \overline{Z}_j\}_{j=1}^{k}$. Up to global phase, every Pauli error can be written as

$$E = T(s)L(\ell)S(a), \tag{2.28}$$

where

$$T(s) = \prod_{i=1}^{n-k} T_i^{s_i}, \quad L(\ell) = \prod_{j=1}^{k} \overline{X}_j^{\ell_j^X} \overline{Z}_j^{\ell_j^Z}, \quad S(a) = \prod_{i=1}^{n-k} S_i^{a_i}. \tag{2.29}$$

with $s, a \in \mathbb{F}_2^{n-k}$, $\ell = \begin{pmatrix} \ell^X \\ \ell^Z \end{pmatrix} \in \mathbb{F}_2^{2k}$. For a fixed canonical symplectic basis, the decomposition is unique modulo global phase.

The three factors have direct operational meanings: (i) $T(s)$ determines the measured syndrome; (ii) $L(\ell)$ determines the logical action on the encoded qubits; and (iii) $S(a)$ selects a stabilizer-equivalent physical representative without affecting the logical state.

Definition 2.2.3 (Syndrome equivalence class)**.** For a fixed syndrome s, the *syndrome equivalence class* is

$$\mathcal{E}(s) := \left\{ E \in \overline{\mathcal{P}}_n : s(E) = s \right\}. \tag{2.30}$$

Using the TLS decomposition,

$$\mathcal{E}(s) = \bigsqcup_{\ell} T(s)L(\ell)\mathcal{S}. \tag{2.31}$$

Thus, errors in $\mathcal{E}(s)$ have the same syndrome but may have different logical components.

Definition 2.2.4 (Logical label class)**.** For a fixed logical label ℓ, define the *logical label class* by

$$\mathcal{E}(\ell) := \bigsqcup_{s} T(s)L(\ell)\mathcal{S}. \tag{2.32}$$

All errors in $\mathcal{E}(\ell)$ have the same logical component ℓ, while their syndromes may differ.

Definition 2.2.5 (Logical coset)**.** For a syndrome s and logical label ℓ, define the *logical coset*, i.e., the logical class at syndrome s by

$$\mathcal{E}(s, \ell) := T(s)L(\ell)\mathcal{S}. \tag{2.33}$$

It contains all errors with syndrome s and logical label ℓ, differing only by stabilizers.

Definition 2.2.6 (Degeneracy in the TLS decomposition)**.** Recall the definition in Definition 2.1.9. Now we can define the degeneracy using the TLS decomposition. Let

$$E_a = T(s_a)L(\ell_a)S(a), \qquad E_b = T(s_b)L(\ell_b)S(b) \tag{2.34}$$

be two Pauli errors expressed in the same TLS basis. They are *degenerate* if they have the same syndrome and logical components,

$$s_a = s_b, \qquad \ell_a = \ell_b, \tag{2.35}$$

while their stabilizer components may differ. Equivalently, $E_a, E_b \in \mathcal{E}(s, \ell)$ for some (s, ℓ), or, equivalently, $E_a^\dagger E_b \in \mathcal{S}$, recovering Definition 2.1.9.

2.3 Decoding Problem

A syndrome does not in general identify a unique physical error: many Pauli errors may produce the same syndrome, and some of them may differ by a non-trivial logical operator. The task of a decoder is therefore to use the observed syndrome together with an assumed noise model to infer a recovery that restores the corrupted state to the original logical state with high probability. This requires more than merely finding an estimate consistent with the syndrome.

Definition 2.3.1 (Syndrome decoder)**.** Given a parity-check matrix H and a syndrome s, a syndrome decoder is a map $\mathcal{D} : s \mapsto \hat{e}$ that returns an error estimate satisfying

$$H\hat{e} = s. \tag{2.36}$$

The corresponding Pauli operator is used as the recovery, or equivalently tracked as a Pauli-frame update [29].

For a fixed syndrome, the syndrome equation generally admits many solutions.

Proposition 2.3.2 (Affine solution space)**.** *Let e_0 be any particular solution of $He = s$. Then every solution is of the form*

$$e = e_0 + v, \quad v \in \ker H, \tag{2.37}$$

and therefore, under the binary representation, the syndrome equivalence class defined in Eq. (2.30) is the affine space

$$\mathcal{E}(s) = \{e \in \mathbb{F}_2^{2n} : He = s\} = e_0 + \ker H. \tag{2.38}$$

Proof. If e and e_0 both satisfy the syndrome equation, then $H(e + e_0) = s + s = 0$, so $e + e_0 \in \ker H$. Conversely, adding any kernel vector to e_0 leaves the syndrome unchanged. □

For a stabilizer code, $\ker H$ is the binary representation of the normalizer $\mathcal{N}(\mathcal{S})$. It contains both the stabilizer subspace $\mathrm{row}(G_\mathcal{S})$, i.e., the subspace spanned by the stabilizer generators, and vectors representing non-trivial logical operators.

Proposition 2.3.3 (Syndrome consistency versus logical success)**.** *Suppose $e, \hat{e} \in \mathcal{E}(s)$. Then the residual error satisfies*

$$r = e + \hat{e} \in \ker H. \tag{2.39}$$

The recovery is logically successful if and only if e and $\hat{e}$ belong to the same logical coset, equivalently if and only if $r \in \mathrm{row}(G_\mathcal{S})$.

Proof. Since $He = H\hat{e} = s$, we have $H(e+\hat{e}) = s+s = 0$. So $r = e+\hat{e} \in \ker H$. The second statement follows directly from Definition 2.2.5, since two errors belong to the same coset exactly when they differ by a stabilizer. □

Consequently,

$$H\hat{e} = s \quad \not\Rightarrow \quad e + \hat{e} \in \mathrm{row}(G_\mathcal{S}). \tag{2.40}$$

A decoder may therefore return a syndrome-consistent estimate while leaving a non-trivial residual logical operator.

The ambiguity within $\mathcal{E}(s)$ leads to two related maximum-likelihood decoding objectives. A decoder may either select the most likely individual error or exploit degeneracy by comparing the total probability of entire cosets [16, 15].

Definition 2.3.4 (Quantum maximum-likelihood decoding)**.** Quantum maximum-likelihood decoding (QMLD) selects the most likely individual error consistent with the syndrome:

$$\hat{e}_{\mathrm{QMLD}} = \arg\max_{e \in \mathcal{E}(s)} P(e \mid s). \tag{2.41}$$

The associated decision problem is NP-complete [16].

Definition 2.3.5 (Degenerate quantum maximum-likelihood decoding)**.** A key difference between classical and quantum error correction is degeneracy. Degenerate quantum maximum-likelihood decoding (DQMLD) selects the most-likely logical coset:

$$\hat{\ell}_{\mathrm{DQMLD}} = \arg\max_{\ell \in \mathbb{F}_2^{2k}} P(\ell \mid s) = \arg\max_{\ell \in \mathbb{F}_2^{2k}} \sum_{e \in \mathcal{E}(s,\ell)} P(e \mid s). \tag{2.42}$$

Exact degenerate maximum-likelihood decoding is #P-hard [16].

2.4 Tanner Graph

A sparse binary matrix is not only an algebraic description of parity constraints, but also defines a graphical model called a Tanner graph. This representation exposes which variables participate in each constraint and provides the computational structure used by iterative decoders such as belief propagation [30]. This graphical representation will be used throughout the following sections to describe the constraints underlying the decoding problem.

Definition 2.4.1 (Tanner graph)**.** Let $H \in \mathbb{F}_2^{m \times n}$ be a binary parity-check matrix. The *Tanner graph* associated with H is the bipartite graph $G = (V \sqcup C, E)$, where

- $V = \{v_1, \ldots, v_n\}$ is the set of *variable nodes*;
- $C = \{c_1, \ldots, c_m\}$ is the set of *check nodes*;
- $(c_i, v_j) \in E$ if and only if $H_{ij} = 1$.

The two parts of the bipartition are disjoint, and every edge joins exactly one variable node to one check node.

For a variable node v_j and a check node c_i, define the neighborhoods

$$\mathcal{N}(v_j) := \{c_i : H_{ij} = 1\}, \qquad \mathcal{N}(c_i) := \{v_j : H_{ij} = 1\}. \tag{2.43}$$

Their degrees are $\deg(v_j) = |\mathcal{N}(v_j)|$ and $\deg(c_i) = |\mathcal{N}(c_i)|$, respectively.

Definition 2.4.2 (LDPC representation)**.** A family of Tanner graphs is *low-density* if the variable and check degrees remain bounded, or grow much more slowly than the block length, as the code size increases [11]. A (d_v, d_c)-regular Tanner graph has $\deg(v_j) = d_v$ and $\deg(c_i) = d_c$ for every variable node v_j and check node c_i, respectively [32].

Proposition 2.4.3 (Graph constraints and the syndrome equation)**.** *Let $e \in \mathbb{F}_2^n$ be a binary error vector and let $s \in \mathbb{F}_2^m$. The syndrome equation $He = s$ holds if and only if every check node c_i satisfies the local parity constraint*

$$\bigoplus_{j \in \mathcal{N}(c_i)} e_j = s_i, \tag{2.44}$$

where $\oplus$ denotes addition modulo two.

Proof. The ith component of He is $(He)_i = \sum_{j=1}^{n} H_{ij} e_j = \bigoplus_{j \in \mathcal{N}(c_i)} e_j$. Hence $He = s$ if and only if the local parity constraint Eq. (2.44) is satisfied for every check node c_i. □

2.5 Noise Models

The performance of a quantum error-correcting code and decoder strongly depends on the assumed noise model. Three commonly used models, with increasing levels of physical realism, are the *code-capacity*, *phenomenological*, and *circuit-level* noise models.

Code-capacity noise. In the code-capacity model, errors are applied only to the data qubits, while syndrome measurements are assumed to be perfect. A typical example is independent Pauli noise acting on each data qubit before an ideal round of syndrome extraction. This model isolates the intrinsic error-correcting capability of the code and is therefore useful for studying decoder behaviour without additional complications from noisy syndrome extraction.

Phenomenological noise. The phenomenological model additionally includes errors in syndrome measurements. Data-qubit errors and measurement errors are usually modeled as independent stochastic events, and several rounds of syndrome extraction are performed in order to distinguish data errors from measurement errors. Although more realistic than the code-capacity model, it does not explicitly model the physical syndrome-extraction circuit.

Circuit-level noise. The circuit-level model introduces noise directly at the level of the syndrome-extraction circuit. Faults may occur during state preparation, gates, measurements, and, depending on the model, idle operations. A single physical fault can therefore propagate through the circuit and produce correlated errors on multiple qubits. Circuit-level noise provides a more realistic setting for fault-tolerant quantum error correction, but also leads to a more complicated decoding problem.

Noise Source	**Code-capacity**	**Phenomenological**	**Circuit-level**
Data-qubit errors	✓	✓	✓
Measurement errors	–	✓	✓
Gate errors	–	–	✓
Preparation errors	–	–	✓
Idle errors	–	–	Model dependent

Table 2.1: Noise sources included in different quantum error correction noise models.

2.5.1 Detector Error Model

As summarized in Table 2.1, under circuit-level noise, faults may occur at different locations of the syndrome-extraction circuit. Their effects depend not only on the measured stabilizers, but also on how errors propagate through the particular circuit implementation. Consequently, two circuits measuring the same stabilizers may induce different classical decoding problems. A *detector error model* (DEM) provides a compact description of this circuit-level noise in terms of the detector events and logical-observable flips produced by elementary error mechanisms [12, 9].

Let $m \in \mathbb{F}_2^m$ be the binary measurement record. Individual measurement outcomes need not be deterministic even for an ideal circuit: the same measurement may yield $m_i = 0$ in one execution but $m_i = 1$ in another. A *detector* is instead a parity of measurement outcomes whose value is deterministic in the absence of faults. Let $D \in \mathbb{F}_2^{d\times m}$ be the matrix whose rows specify these detector parities, and let b denote their ideal parity values. The detector event vector is

$$s := Dm + b, \qquad Dm^{(0)} = b \tag{2.45}$$

for every noiseless record $m^{(0)}$. Thus a detector is triggered when the corresponding component of s equals one.

Let $e_1, \ldots, e_n$ be the error mechanisms compressed from the original circuit faults and let $e \in \mathbb{F}_2^n$ be the error configuration indicating which error mechanisms occur.

Definition 2.5.1 (Error weight)**.** The *error weight* of an error configuration e is its Hamming weight,

$$\mathrm{wt}(e) := |\{j : e_j = 1\}| = \sum_{j=1}^{n} e_j. \tag{2.46}$$

It counts the number of activated error mechanisms, rather than the number of data qubits on which the resulting Pauli error acts non-trivially.

If $\Omega \in \mathbb{F}_2^{m\times n}$ records which measurements each error mechanism flips, then

$$m = m^{(0)} + \Omega e. \tag{2.47}$$

Substitution into Eq. (2.45) gives

$$s = He, \tag{2.48}$$

where $H := D\Omega$ is the detector error matrix. The jth column of H is therefore the detector pattern caused by the single error mechanism e_j.

The detector error matrix H directly defines a Tanner graph: variable nodes represent error mechanisms, check nodes represent detectors, and $H_{ij} = 1$ places an edge between error mechanism e_j and detector i. The algebra has the same form as a parity-check matrix, but the variables represent circuit-level error mechanisms rather than Pauli error on data qubits. Accordingly, in the subsequent chapters, we also refer to H as the parity-check matrix. Circuit-level decoding can therefore use the same iterative decoders such as belief propagation, with the error mechanism probabilities supplying the variable priors.

To track logical failure, introduce a matrix L such that Le records the logical observables flipped by e. A DEM can then be summarized as a triple (H, L, π), where $\pi(e)$ is the probability distribution of the error mechanisms. It is often listed column by column as $(H_{\cdot j}, L_{\cdot j}, \pi_j)$ when the mechanisms are independent; correlated mechanisms instead require a joint prior or a combined elementary event. Given an estimate $\hat{e}$ with $H\hat{e} = s$, decoding is *logically successful* precisely when

$$L(e + \hat{e}) = 0. \tag{2.49}$$

Definition 2.5.2 (Circuit distance)**.** Under circuit-level noise, we can define the *circuit distance* d_{circ}, which is similar to code distance d, as the minimum number of error mechanisms required to produce a non-trivial logical effect,

$$d_{\text{circ}} := \min_e \{\text{wt}(e) \ : \ He = 0, \ Le \neq 0\}. \tag{2.50}$$

The circuit distance depends on the syndrome-extraction circuit and noise model, and therefore need not in general coincide with the code distance. However, we find that the two coincide in our experiments. Therefore, we do not distinguish between them in the subsequent chapters.

2.6 Waterfall and Error-floor Region

The performance of a code-decoder pair is commonly characterized by its logical error rate as a function of the physical noise parameter. The resulting curve often exhibits two qualitatively different regions: a *waterfall region*, where the logical error rate decreases rapidly as the physical noise is reduced, and a low-noise *error-floor region*, where the performance is dominated by rare, often low-weight, errors [17].

2.6.1 Logical Error Rate and Threshold Behaviour

Definition 2.6.1 (Logical error rate)**.** Fix a code $\mathcal{C}$, a decoder $\mathcal{D}$, a syndrome-extraction procedure, and a noise model parameterized by a physical error rate p. The *logical error rate* $P_{\text{L}}(p; \mathcal{C}, \mathcal{D})$ is the probability that the residual error after recovery induces a non-trivial logical operation on the encoded state.

Operationally, the logical error rate is estimated by repeatedly sampling errors from the noise model, decoding the syndrome, and checking whether the residual error is logically non-trivial. If N independent samples are generated and N_{fail} of them result in a logical failure, the empirical logical error rate is

$$\widehat{P}_{\text{L}} = \frac{N_{\text{fail}}}{N}. \tag{2.51}$$

Definition 2.6.2 (Decoding Threshold)**.** Consider a family of codes $\{\mathcal{C}_d\}$ with increasing distance, a

corresponding decoder, and a fixed noise model. A *threshold* p_{th} is a critical physical error rate such that, for $p < p_{\mathrm{th}}$, the logical error rate can be made arbitrarily small by increasing the distance, while this asymptotic suppression is lost for $p > p_{\mathrm{th}}$. For finite distances, p_{th} is typically estimated from crossings of logical-error-rate curves or by finite-size scaling [8].

Definition 2.6.3 (Waterfall region)**.** The *waterfall region* denotes the range of physical error rates p over which the logical error rate $P_{\mathrm{L}}(p)$ decreases rapidly as p is reduced. For a finite code family, this region typically appears around the finite-size crossover associated with the decoding threshold [1, 28].

2.6.2 Dominant Low-Weight Failures

At sufficiently low physical error rates, the logical error rate is often controlled by the lowest-weight error configurations that can lead to logical failure. Weight-enumerator methods make this principle explicit: the distribution of harmful errors by weight determines the leading low-noise behaviour of the logical error rate [10]. Similarly, for surface codes, Regev et al. count the number of physical error configurations of each weight that result in logical failure and use these counts to approximate the logical error rate [27].

To formulate the same idea for a general system, consider a fixed finite set of n error mechanisms. For a fixed deterministic decoder $\mathcal{D}$, let $e \in \mathbb{F}_2^n$ denote the error configuration, and let $\mathsf{Fail}_{\mathcal{D}}(e) \in \{0, 1\}$ indicate whether decoding results in a logical failure or not, according to the criterion introduced in Eq. (2.49).

Definition 2.6.4 (Minimum failing weight)**.** For each weight w, define

$$A_w(\mathcal{D}) := \left|\{e \in \mathbb{F}_2^n : \mathrm{wt}(e) = w,\ \mathsf{Fail}_{\mathcal{D}}(e) = 1\}\right|. \tag{2.52}$$

Thus, $A_w(\mathcal{D})$ counts the weight-w error configurations that cause logical failure for the chosen system. The *minimum failing weight* is

$$w_*(\mathcal{D}) := \min\{w : A_w(\mathcal{D}) > 0\} = \min\{\mathrm{wt}(e) : \mathsf{Fail}_{\mathcal{D}}(e) = 1\}. \tag{2.53}$$

It is the minimum number of error mechanisms capable of causing a logical failure.

Remark 2.6.5 (Relation to code distance)**.** The code distance d is a property of the code, whereas w_* characterizes the complete code–noise–decoder system. It may therefore depend on the syndrome-extraction circuit, the noise model, and the decoder. Under ideal syndrome extraction and optimal decoding, the code distance constrains the minimum weight of harmful data errors. Under circuit-level noise or suboptimal decoding, however, logical failure may occur at lower error weight because of fault propagation or decoder-specific failure mechanisms. Hence w_* should not, in general, be identified with d. Conceptually, w_* is closer to the critical number considered in trapping set analysis [26].

Proposition 2.6.6 (Low-noise asymptotic behaviour)**.** *Assume that the n binary error mechanisms occur independently with an identical probability p, and that $\mathcal{D}$ is a fixed, p-independent deterministic decoder. Then*

$$P_{\mathrm{L}}(p) = \sum_{w=0}^{n} A_w(\mathcal{D})\, p^w (1-p)^{n-w}. \tag{2.54}$$

Consequently,

$$P_{\mathrm{L}}(p) = A_{w_*}(\mathcal{D})\, p^{w_*} + O\big(p^{w_*+1}\big), \qquad p \to 0. \tag{2.55}$$

Proof. Every weight-w error configuration occurs with probability $p^w(1-p)^{n-w}$, and $A_w(\mathcal{D})$ such configurations lead to logical failure. Since $A_w(\mathcal{D}) = 0$ for all $w < w_*$, the first non-zero term occurs at $w = w_*$. □

Corollary 2.6.7 (Asymptotic log–log slope)**.** *Define the local log–log slope by*

$$\gamma(p) := \frac{d \log P_{\mathrm{L}}(p)}{d \log p}. \tag{2.56}$$

Under the assumptions of Proposition 2.6.6,

$$\lim_{p\to 0} \gamma(p) = w_*. \tag{2.57}$$

Hence the leading low-noise behaviour is characterized by two quantities: w_* determines the asymptotic power exponent, while A_{w_*} determines the multiplicity of the dominant minimum-weight failures. Eliminating only part of this failure set reduces the leading coefficient, whereas eliminating all weight-w_* failures increases the leading exponent.

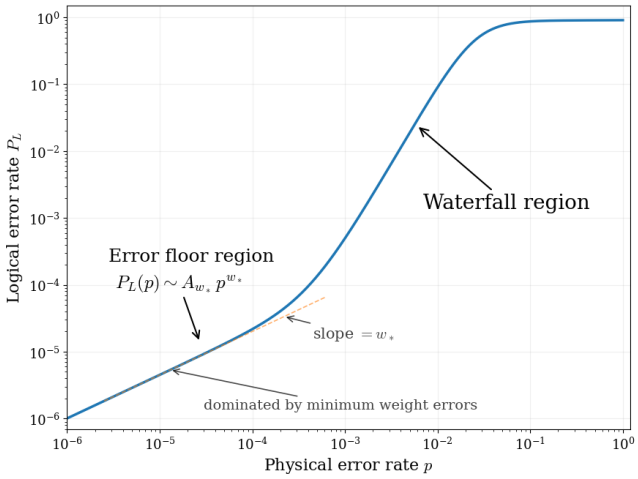


Figure 2.2: The logical error rate curve illustrating the waterfall and error-floor region. At moderate physical error rates, the logical error rate decreases rapidly in the waterfall region. At sufficiently low p, the behaviour is instead dominated by the minimum-weight failing error configurations, giving the asymptotic form $P_L(p) \sim A_{w_*} p^{w_*}$, where w_* is the minimum failing weight and A_{w_*} its multiplicity. The corresponding log–log slope therefore approaches w_* in the error-floor region.

The preceding result assumes identical probabilities. In a detector error model, however, different error mechanisms generally have different priors. The same low-noise argument can be extended by considering the asymptotic order of each error mechanism.

Definition 2.6.8 (DEM failure order)**.** Consider a detector error model with n independent error mechanisms, where mechanism j occurs with probability $p_j(p)$. Suppose that, as $p \to 0$,

$$p_j(p) = c_j p^{\nu_j} + o(p^{\nu_j}), \qquad c_j > 0, \quad \nu_j > 0. \tag{2.58}$$

For an error configuration $e \in \mathbb{F}_2^n$, define its *asymptotic error order* by $\omega(e) := \sum_{j=1}^n \nu_j e_j$. The *minimum failing order* is

$$\omega_* := \min_{\mathsf{Fail}(e)=1} \omega(e). \tag{2.59}$$

In the circuit-level noise models considered in this work, the relevant DEM mechanisms are typically first order in the physical error rate, $p_j(p) = c_j p + O(p^2)$. Hence $\nu_j = 1$ and, in practice, $\omega(e) = \mathrm{wt}(e)$. The non-uniform DEM priors therefore affect the relative weights of different configurations, but not their leading asymptotic order.

Proposition 2.6.9 (Low-noise asymptotics for a detector error model)**.** *Assume that the DEM error mechanisms are independent and that the decoder defines a fixed, p-independent deterministic mapping in a neighbourhood of $p = 0$. Then*

$$P_{\mathrm{L}}(p) = C_{\omega_*} p^{\omega_*} + o(p^{\omega_*}), \qquad p \to 0, \tag{2.60}$$

where

$$C_{\omega_*} = \sum_{\substack{e:\,\mathsf{Fail}(e)=1 \\ \omega(e)=\omega_*}} \prod_{j:e_j=1} c_j. \tag{2.61}$$

Proof. For a configuration e, independence gives

$$\Pr(e) = \prod_{j=1}^{n} p_j(p)^{e_j} \big(1 - p_j(p)\big)^{1-e_j}. \tag{2.62}$$

Using the low-noise expansion of $p_j(p)$,

$$\Pr(e) = \left(\prod_{j:e_j=1} c_j \right) p^{\omega(e)} + o\Big(p^{\omega(e)}\Big). \tag{2.63}$$

Summing over all failing configurations, the leading contribution comes from those with the minimum order ω_*. □

Definition 2.6.10 (Error-floor region)**.** The *error-floor region* is the low-noise region in which the logical error rate is dominated by a relatively small set of harmful configurations, typically of low weight or low asymptotic order, rather than by the broader distribution of errors characteristic of the waterfall region [28, 26].

3 Quantum Error Correcting Codes

This chapter introduces the code needed for the decoding analysis. We first review Calderbank–Shor–Steane (CSS) codes, which express stabilizer constraints through binary parity-check matrices and thereby connect the stabilizer formalism of the previous chapter to classical decoding methods. We then describe quantum low-density parity-check (qLDPC) codes, whose sparse checks make message-passing decoders such as belief propagation computationally attractive. Finally, we present the triangular code family used in the experiments.

3.1 CSS Code

Calderbank–Shor–Steane (CSS) codes are an important class of stabilizer quantum error-correcting codes constructed from two classical binary linear codes, with stabilizer generators separated into X-type and Z-type checks [7]. Well-known examples include surface codes [8] and hypergraph-product codes [33]. This separation allows the X and Z errors to be treated using classical parity-check matrices and, in many noise models, decoded separately.

Definition 3.1.1 (CSS check matrices)**.** Let $H_X \in \mathbb{F}_2^{r_X \times n}$ and $H_Z \in \mathbb{F}_2^{r_Z \times n}$ be two binary parity-check matrices satisfying

$$H_X H_Z^T = 0 \pmod 2. \tag{3.1}$$

For binary vectors $x = (x_1, \ldots, x_n)$ and $z = (z_1, \ldots, z_n)$, define

$$X^x := \prod_{j=1}^{n} X_j^{x_j}, \qquad Z^z := \prod_{j=1}^{n} Z_j^{z_j}. \tag{3.2}$$

The rows of H_X specify X-type stabilizer generators, while the rows of H_Z specify Z-type stabilizer generators.

Proposition 3.1.2 (CSS commutation condition)**.** *The X- and Z-type stabilizer generators commute if and only if Eq. (3.1) holds.*

Proof. For binary vectors $x, z \in \mathbb{F}_2^n$, the Pauli operators X^x and Z^z satisfy

$$X^x Z^z = (-1)^{xz^T} Z^z X^x. \tag{3.3}$$

Thus every row of H_X must have even overlap with every row of H_Z, which is exactly the condition $H_X H_Z^T = 0$ over $\mathbb{F}_2$. □

Definition 3.1.3 (CSS stabilizer group)**.** The stabilizer group generated by H_X and H_Z is

$$\mathcal{S} := \langle X^x Z^z \mid x \in \mathrm{row}(H_X),\ z \in \mathrm{row}(H_Z) \rangle\,. \tag{3.4}$$

Equivalently, its binary symplectic generator matrix is

$$G_{\mathcal{S}} := \begin{bmatrix} H_X & 0 \\ 0 & H_Z \end{bmatrix}. \tag{3.5}$$

Proposition 3.1.4 (Number of encoded qubits)**.** *A CSS code defined by H_X and H_Z encodes*

$$k = n - \mathrm{rank}(H_X) - \mathrm{rank}(H_Z) \tag{3.6}$$

logical qubits from n physical qubits.

Proof. From the stabilizer generator matrix $G_{\mathcal{S}}$ of CSS code, we have $\mathrm{rank}(G_{\mathcal{S}}) = \mathrm{rank}(H_X) +$

$\mathrm{rank}(H_Z)$. A stabilizer code with r independent generators has codespace dimension 2^{n-r}. Hence

$$2^k = 2^{\,n-\mathrm{rank}(H_X)-\mathrm{rank}(H_Z)}, \tag{3.7}$$

which gives Eq. (3.6). □

Remark 3.1.5 (Redundant checks)**.** The numbers of rows r_X and r_Z need not equal the corresponding ranks. Adding a row that is a binary linear combination of existing rows does not change the stabilizer group or the number of logical qubits, although it changes the Tanner graph and may affect an iterative decoder.

Let a Pauli error be represented as $\begin{pmatrix} x \\ z \end{pmatrix} \in \mathbb{F}_2^{2n}$. Using Eq. (3.5), its syndrome separates as

$$s_X = H_X z^T, \qquad s_Z = H_Z x^T. \tag{3.8}$$

This separation is exact at the level of stabilizer commutation. A physical noise model may nevertheless correlate x and z, as occurs for Pauli-Y errors or correlated circuit faults.

Definition 3.1.6 (CSS logical spaces and asymmetric distances)**.** The binary supports of non-trivial logical X and Z operators are represented by the quotient spaces

$$\mathcal{L}_X := \ker(H_Z)/\mathrm{row}(H_X), \qquad \mathcal{L}_Z := \ker(H_X)/\mathrm{row}(H_Z). \tag{3.9}$$

The corresponding distances are

$$d_X := \min_{x \in \ker(H_Z)\setminus \mathrm{row}(H_X)} \mathrm{wt}(x), \tag{3.10}$$

$$d_Z := \min_{z \in \ker(H_X)\setminus \mathrm{row}(H_Z)} \mathrm{wt}(z). \tag{3.11}$$

The usual stabilizer distance is $d = \min(d_X, d_Z)$.

Remark 3.1.7 (Independent versus joint decoding)**.** For independent X- and Z-noise models, one may decode the two equations in Eq. (3.8) separately. Under depolarizing or circuit-level noise, this can discard correlations between the two components; a joint decoder may then outperform two independent binary decoders.

3.2 Quantum LDPC Code

Quantum low-density parity-check (qLDPC) codes are quantum error-correcting codes whose stabilizer checks have bounded weight and whose qubits participate in only a bounded number of checks. More concretely, for a family of stabilizer codes with parameters $[\![n, k, d]\!]$, the code is called qLDPC if the weights of the stabilizer generators remain $O(1)$ as n grows, and each physical qubit is involved in only $O(1)$ stabilizer checks [23, 6].

Although qLDPC codes can have much better asymptotic parameters than the surface code, they are usually harder to implement physically. The reason is that their Tanner graphs are not necessarily geometrically local in two dimensions. As a result, measuring their stabilizers may require long-range couplings, qubit shuttling, modular links, or other hardware mechanisms beyond nearest-neighbor planar connectivity. Therefore, the usefulness of a qLDPC code depends not only on its abstract parameters $[\![n, k, d]\!]$, but also on the syndrome-extraction circuit, the available hardware connectivity, and the performance of the decoder [31, 36].

3.2.1 Triangular Code

The triangular code used in this work belongs to the broad family of classical cellular automaton codes, whose code structure is generated by iterating local cellular automaton update rules [31, 5, 21]. The triangular code is defined directly by local parity constraints on a growing triangular space

lattice. All cells generated during the evolution are retained as code coordinates. This family appears as a local phase-flip code in the LDPC-cat architecture of Ruiz et al. [31] and is analysed explicitly as a decoder benchmark by Benois et al. [4].

Definition 3.2.1 (Triangular code family)**.** The family is indexed by an integer $\ell \geq 1$, where ℓ is the number of stabilizer layers. Its members are denoted by $\mathcal{T}_\ell$ and have classical parameters

$$\mathcal{T}_\ell = [n(\ell), 2, d(\ell)]\,, \qquad n(\ell) = \frac{(\ell+2)(\ell+3)}{2} - 1, \qquad d(\ell) = G_{\ell+1}, \tag{3.12}$$

where $\{G_\ell\}_{\ell\in\mathbb{N}}$ denotes the partial sums of Gould's sequence [22]. Each successive row contains one more data qubit than the row below it. Extending the lattice by one row introduces two-body Z-checks at the two boundaries and three-body Z-checks on the triangular faces in the bulk [4].

The origin of the two logical degrees of freedom can be seen directly from the local parity constraints. Let $x_i \in \mathbb{F}_2$ indicate whether an X-type operator acts on data qubit i. A two-body boundary check imposes $x_i = x_j$ on neighbouring boundary qubits, while a triangular check with vertices i, j, k imposes

$$x_k = x_i + x_j \pmod 2. \tag{3.13}$$

Consequently, choosing the values on the two bottom boundary qubits determines the entire operator support row by row. The four choices in $\mathbb{F}_2^2$ give the identity and the three products of the two logical X operators.

Remark 3.2.2 (Fractal logical support)**.** The recursion in Eq. (3.13) reproduces Pascal's triangle modulo two. The non-zero entries therefore form a Sierpiński-triangle pattern, and the weights of the minimum logical operators are governed by Gould's sequence.

The small triangular examples below are naturally described by Z-type checks only. They are useful for studying the decoding of X errors, but they should not be presented as distance-3 or distance-5 *full quantum codes*.

Example 3.2.3 ($\mathcal{T}_1 = [5, 2, 3]$ triangular code)**.** The Z-type generators are

$$\mathcal{S}_Z = \langle Z_1Z_3, Z_1Z_2Z_4, Z_2Z_5 \rangle\,, \tag{3.14}$$

with the logical operators

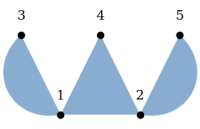
$$(\overline{X}_1, \overline{X}_2) = (X_1X_3X_4, X_2X_4X_5), \qquad (\overline{Z}_1, \overline{Z}_2) = (Z_1, Z_2). \tag{3.15}$$

Example 3.2.4 ($\mathcal{T}_2 = [9, 2, 5]$ triangular code)**.** The Z-type generators are

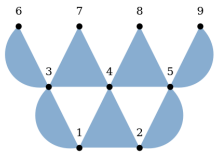
$$\mathcal{S}_Z = \langle Z_1Z_3, Z_1Z_2Z_4, Z_2Z_5, Z_3Z_6, Z_3Z_4Z_7, Z_4Z_5Z_8, Z_5Z_9 \rangle. \tag{3.16}$$

with the logical operators

$$(\overline{X}_1, \overline{X}_2) = (X_1X_3X_4X_6X_8, X_2X_4X_5X_7X_9), \qquad (\overline{Z}_1, \overline{Z}_2) = (Z_1, Z_2). \tag{3.17}$$



(a) $\mathcal{T}_1 = [5, 2, 3]$ triangular code.



(b) $\mathcal{T}_2 = [9, 2, 5]$ triangular code.

Figure 3.1: Geometric representations of the two triangular codes used in this work. Black vertices denote data qubits, labelled by their indices. Each blue region represents the support of a Z-type stabilizer generator: boundary regions correspond to two-body checks, while triangular regions correspond to three-body checks.

4 Decoders

4.1 BP Decoder

Belief propagation (BP) is an iterative message-passing algorithm on the Tanner graph associated with a parity-check matrix. Messages are exchanged between variable nodes and check nodes to estimate the marginal error probabilities of the variables. This results in a soft-decision output indicating the likelihood of errors. In CSS codes, the X and Z components are decoded separately [30].

For each variable node v, define the prior error probability by $p_v := \Pr(e_v = 1)$. The corresponding soft information is expressed in terms of the log-likelihood ratio (LLR):

$$\lambda_v := \log \frac{\Pr(e_v = 0)}{\Pr(e_v = 1)} = \log \frac{1 - p_v}{p_v}. \tag{4.1}$$

A positive LLR favors $e_v = 0$, while a negative LLR favors $e_v = 1$. These values quantify the reliability of each bit and are used to guide the subsequent decoding stage.

Let $\mathcal{N}(v)$ denote the set of checks adjacent to variable v, and let $\mathcal{N}(c)$ denote the set of variables adjacent to check c. BP iteratively exchanges messages between variables and checks and updates variables' beliefs. Initialize with $m_{v\to c}^{(0)} = \lambda_v$. For the sum-product version of BP, the check-to-variable update at iteration t with the syndrome bit s_c is

$$m_{c\to v}^{(t)} = 2 \tanh^{-1} \left[(-1)^{s_c} \prod_{v' \in \mathcal{N}(c)\setminus\{v\}} \tanh\left(\frac{m_{v'\to c}^{(t-1)}}{2} \right) \right]. \tag{4.2}$$

Equivalently, the syndrome changes the sign of the check constraint: if $s_c = 0$, the neighbouring variables must have even parity; if $s_c = 1$, they must have odd parity. Because the sum-product rule has high computational complexity, we usually use the min-sum rule, a low-complexity approximation:

$$m_{c\to v}^{(t)} \approx \alpha \left[(-1)^{s_c} \prod_{v' \in \mathcal{N}(c)\setminus\{v\}} \operatorname{sign}\left(m_{v'\to c}^{(t-1)} \right) \right] \min_{v' \in \mathcal{N}(c)\setminus\{v\}} \left| m_{v'\to c}^{(t-1)} \right|, \tag{4.3}$$

where $\alpha > 0$ is a scaling factor and $\alpha = 1$ gives unscaled min-sum.

The variable-to-check update is

$$m_{v\to c}^{(t)} = \lambda_v + \sum_{c' \in \mathcal{N}(v)\setminus\{c\}} m_{c'\to v}^{(t)}. \tag{4.4}$$

After each iteration, the posterior LLR and hard decision are

$$L_v^{(t)} = \lambda_v + \sum_{c \in \mathcal{N}(v)} m_{c\to v}^{(t)}, \tag{4.5}$$

$$\hat{e}_v^{(t)} = \begin{cases} 0, & L_v^{(t)} \geq 0, \\ 1, & L_v^{(t)} < 0. \end{cases} \tag{4.6}$$

The decoder stops when $H\hat{e}^{(t)} = s$ or when the maximum number of iterations is reached.

Algorithm 1 Belief Propagation, sum-product rule

Input: parity-check matrix H, syndrome s, channel LLRs $\{\lambda_i\}_{i=1}^{n}$, maximum number of iterations $T_{\max}$
Initialize: $m_{v\to c}^{(0)} \leftarrow \lambda_v$ for all edges (v,c) in the Tanner graph
for $t = 1$ to $T_{\max}$ **do**
  for each check node c **do**
    for each $v \in N(c)$ **do**
      $m_{c\to v}^{(t)} \leftarrow 2\tanh^{-1}\left[(-1)^{s_c}\prod_{v'\in N(c)\setminus\{v\}}\tanh\left(\frac{m_{v'\to c}^{(t-1)}}{2}\right)\right]$
    end for
  end for
  for each variable node v **do**
    for each $c \in N(v)$ **do**
      $m_{v\to c}^{(t)} \leftarrow \lambda_v + \sum_{c'\in N(v)\setminus\{c\}} m_{c'\to v}^{(t)}$
    end for
  end for
  for each variable node v **do**
    $L_v \leftarrow \lambda_v + \sum_{c\in N(v)} m_{c\to v}^{(t)}$
    $\hat{e}_v \leftarrow 0$ if $L_v \geq 0$ else $\hat{e}_v \leftarrow 1$
  end for
  if $H\hat{e} = s \pmod 2$ **then**
    return $\hat{e}$
  end if
end for
return $\hat{e}$

4.2 BP+OSD Decoder

BP with ordered-statistics decoding (BP+OSD) supplements the BP stage described above with Gaussian-elimination post-processing. When the standard BP stage reports failure, its final soft output is used to choose a likely error basis, and OSD constructs a correction satisfying the syndrome equation by design [23, 30].

Let $P_v := \Pr(e_v = 1 \mid s)$ denote the final BP estimate of the probability that variable v is in error, which can be recovered as $P_v = (1 + e^{L_v})^{-1}$ from posterior LLRs of BP. The variables are ordered from largest to smallest P_v, namely from most to least likely to be in error.

Let $r = \operatorname{rank}(H)$. After permuting the columns of H according to the ordering, select the first r linearly independent columns. Their indices form the *basis set* S, and the remaining indices form the *remainder set* T. Thus

$$H_{[S,T]} = \left[H_{[S]} \mid H_{[T]}\right], \qquad e_{[S,T]} = \left(e_{[S]}, e_{[T]}\right), \tag{4.7}$$

and the syndrome equation becomes

$$H_{[S]}e_{[S]} + H_{[T]}e_{[T]} = s. \tag{4.8}$$

The columns of $H_{[S]}$ form a basis of $\operatorname{Im} H$, i.e., a basis of the syndrome space. Although $H_{[S]}$ need not literally be square when H contains redundant rows, Gaussian elimination provides the unique basis coordinates solving

$$H_{[S]}e_{[S]} = s + H_{[T]}e_{[T]}. \tag{4.9}$$

The simplest post-processing method is OSD-0. It sets the remainder bits to zero, i.e., $e_{[T]} = 0$, and solves

$$H_{[S]}e_{[S]}^{(0)} = s. \tag{4.10}$$

The OSD-0 candidate is therefore $e_{[S,T]}^{(0)} = \left(e_{[S]}^{(0)}, 0\right)$, and always satisfies $He^{(0)} = s$.

Higher-order OSD considers non-zero assignments of the remainder vector $e_{[T]}$. For any chosen $e_{[T]}$, the basis component is

$$e_{[S]} = H_{[S]}^{-1}\left(s + H_{[T]}e_{[T]}\right) \tag{4.11}$$

Every completed vector $e_{[S,T]} = \left(e_{[S]}, e_{[T]}\right)$ is syndrome consistent. Exhaustively searching all $2^{|T|}$ assignments is generally impractical, so some greedy strategy such as combination sweep is used to reduce the complexity [30].

4.3 Complexity Analysis

BP Let n be the number of variables, m the number of checks, and let $|E|$ be the number of edges in the Tanner graph. One BP iteration requires $O(|E|)$ message updates. For LDPC codes, the row and column weights are bounded, so $|E| = O(n)$ and the BP stage costs

$$O(T_{\max} \cdot n). \tag{4.12}$$

OSD The complexity of OSD depends on its order w. The main steps are:

- Sorting reliabilities: $O(n \log n)$,
- Gaussian elimination on an $m \times n$ binary matrix: $O(mn \min(m, n))$ with dense elimination, $O(n^3)$ in the square dense case (it can be reduced in sparse implementations),
- Exhaustive enumeration of error patterns for order-w OSD: $O\left(\sum_{j=0}^{w} \binom{k}{j}\right)$, while combination sweep: $O\left(k + \binom{\lambda}{2}\right)$, where $k = n - \mathrm{rank}(H)$ is the length of $e_{[T]}$, and $\lambda \leq k$ denotes the number of least reliable free variables retained in the combination sweep.

Overall Complexity The overall complexity is therefore

$$O(T_{\max} \cdot n) + O(n^3) + O\left(\sum_{j=0}^{w} \binom{k}{j}\right). \tag{4.13}$$

For small OSD order, especially $w = 0, 1, 2, 3$, the method is tractable. However, the enumeration term grows rapidly with w, so OSD is used as a bounded post-processing stage, where the search relies on restricted candidate-generation strategies such as combination sweep, rather than exhaustive enumeration over all possible patterns.

5 BP Failure Mechanisms

The Tanner graph introduced in Section 2.4 provides the computational structure on which BP operates. Whether BP performs exact inference or only an approximation depends strongly on the local topology of this graph. Trees permit an exact recursive decomposition, whereas cycles allow information to return to its source and be counted repeatedly. This distinction provides the structural starting point for the trapping set and oscillation mechanisms developed later in this chapter.

5.1 Cycles

Definition 5.1.1 (Cycle and girth)**.** A *cycle* is a closed path with no repeated vertices other than its initial and final vertex. The *girth* $g(G)$ is the length of the shortest cycle in G, with $g(G) = \infty$ when G is acyclic. Since Tanner graphs are bipartite, every cycle has even length.

Proposition 5.1.2 (Exactness of BP on trees)**.** *For the posterior factorization in Eq.* (A.9)*, if the Tanner graph is a tree, BP computes the exact single-variable posterior marginals after a finite number of message updates.*

Remark 5.1.3 (Why loops matter)**.** On a graph containing cycles, information may return to its source through another path, so incoming messages are no longer conditionally independent as in the tree. Loopy BP must therefore be regarded as an approximation. The presence of a short cycle alone, however, does not imply decoding failure; the outcome also depends on the syndrome, priors, update schedule, and surrounding graph structure.

Cycles supply the feedback paths that can sustain BP failures, while trapping sets identify the particular local structures for which this feedback becomes harmful.

5.2 Trapping Sets

For finite-length qLDPC codes, iterative decoders such as BP may fail to converge even when the physical error rate is low. Such failures are often caused by small harmful subgraphs in the Tanner graph, known as *trapping sets*. In the quantum case, the situation is more subtle than in classical LDPC codes, because the decoder only observes the error syndrome and because different Pauli errors may be equivalent up to a stabilizer [26].

In the following, we write Tanner graph $G = (V \cup C, E)$, where V is the set of variable nodes and C is the set of check nodes. For a subset $\mathcal{T} \subseteq V$, we denote by $G(\mathcal{T})$ the subgraph induced by $\mathcal{T}$ and its neighbouring check nodes.

Definition 5.2.1 (Quantum trapping set)**.** A *quantum trapping set* (QTS) is a non-empty set of variable nodes $\mathcal{T}_s \subseteq V$ such that, under a given syndrome-based iterative decoder, either the variables in $\mathcal{T}_s$ do not eventually converge or some neighbouring checks are not eventually satisfied. If the induced subgraph $G(\mathcal{T}_s)$ contains a variable nodes and b unsatisfied check nodes, the QTS is denoted by (a, b).

This definition generalizes the classical notion of trapping sets to the syndrome decoding setting. The parameter a measures the size of the harmful variable-node configuration, while b measures the number of unsatisfied checks left by the decoder. Small values of a are particularly important in the low physical error rate region, since low-weight error patterns dominate the error-floor behaviour of finite-length qLDPC codes.

Definition 5.2.2 (Critical number and strength)**.** The *critical number* $\mu(\mathcal{T}_s)$ is the smallest number of initially erroneous variable nodes that can cause the decoder to fail through the trapping set $\mathcal{T}_s$. Its *strength* $\sigma(\mathcal{T}_s)$ is the number of failure-inducing error patterns of weight $\mu(\mathcal{T}_s)$ [26].

Remark 5.2.3 (Harmfulness in the error-floor region)**.** A trapping set with a smaller critical number or a larger strength is typically more harmful. The smallest critical number controls the lowest error weight at which the structure can contribute, while the strength contributes to the multiplicity of such failures. This is the trapping-set interpretation of the low-weight asymptotic behaviour discussed previously.

5.2.1 Classical-type Trapping Sets

Definition 5.2.4 (Classical-type trapping set)**.** A *classical-type trapping set* is a QTS that behaves analogously to a trapping set in a classical LDPC code. It arises from a small subgraph whose message-passing dynamics prevent the decoder from reaching a syndrome-consistent estimate. For example, the decoder may oscillate between two incorrect estimates or converge to an estimate whose syndrome does not match the input syndrome [26].

Such failures are associated with the loopy feedback mechanisms discussed in Section 5.1. Their harmfulness is decoder dependent: the subgraph topology alone does not imply decoding failure.

5.2.2 Symmetric Stabilizer Trapping Sets

The second class of QTSs is specific to quantum codes and has no direct classical analogue. It is related to degeneracy: in a stabilizer code, two errors e and f are degenerate if their difference is a stabilizer. Such errors have the same effect on the encoded quantum state and should be considered equivalent.

However, iterative decoders do not always exploit this degeneracy correctly. In some cases, the Tanner graph contains a stabilizer whose variable nodes can be partitioned into two or more symmetric subsets. If two degenerate error patterns are supported on these symmetric subsets, then a flooding-schedule decoder may receive indistinguishable messages from both sides of the structure. The decoder may then try to converge to both degenerate errors simultaneously and fail to produce an estimate with the correct syndrome. Such a harmful configuration is called a *symmetric stabilizer trapping set* [26].

Definition 5.2.5 (Symmetric stabilizer trapping set)**.** A *symmetric stabilizer trapping set* is supported on a stabilizer whose variable nodes can be partitioned into an even number of disjoint subsets satisfying (i) the subgraphs induced by these subsets are isomorphic; and (ii) each subset has the same set of odd-degree check-node neighbours in the induced subgraph.

5.3 Oscillating Region

During a BP run, some variable nodes may repeatedly change their posterior beliefs and hard decisions instead of settling to stable beliefs. In this work, we use the term *oscillating region* to describe a local Tanner subgraph identified dynamically from a BP trajectory. It consists of variables whose posterior beliefs or hard decisions remain unstable across iterations, together with their neighbouring checks.

Such regions may arise from the loopy feedback and local symmetries discussed above, but their presence must be inferred from the decoding trajectory rather than from graph topology alone. Unlike a trapping set, which describes a potentially harmful static structure, an oscillating region is identified from the trajectory of a particular decoding instance and may change with the syndrome, priors, update schedule, and decoder parameters.

Persistent oscillation can lead to two different outcomes. First, the decoder may fail to converge to a syndrome-consistent estimate before the maximum iteration is reached. Second, it may eventually converge, or temporarily settle, on a syndrome-consistent estimate belonging to the wrong logical class. Oscillation is therefore an indicator of unstable local inference, but its presence alone does not imply logical failure.

6 Experiment

6.1 Experiment Methodology

The experiments were developed iteratively around the central observation: in the restricted circuit-level noise setting, the logical error rate (LER) curve of BP+OSD lies above that of Tesseract in the low physical error rate region. The objective is to identify which stage of BP+OSD creates the additional failures and then test interventions that address the responsible mechanisms. The experiments evolved through four questions:

Q1. Where does the discrepancy appear? Which noise level, physical error rates, and low-weight errors generate the discrepancy?

Q2. Which stage fails? Is the discrepancy associated with BP scheduling, non-convergence, early stopping, unreliable belief, or an insufficient OSD search?

Q3. What mechanisms underlie BP failures? Are they associated with message instability, stable but logically wrong states, or sensitivity to the chosen parity-check representation?

Q4. Which intervention actually repairs each mechanism? The tested interventions include longer BP, restarts, min-sum scaling, redundant-check augmentation, and multiple equivalent parity-check matrices.

Throughout this chapter, we distinguish between *samples* and *instances*. A *sample* refers to an error configuration obtained by Monte Carlo sampling from the specified noise model, whereas an *instance* refers to a configuration obtained by exhaustive enumeration of error mechanisms.

6.1.1 Simulation Settings

Unless stated separately, the general parameters for the BP+OSD decoder in the experiments are listed in Table 6.1.

Parameter	Baseline Settings
Syndrome rounds	one round
Noise source	circuit-level but no measurement flip
BP update rule	min-sum
BP schedules	flooding
Min-sum scaling	$\alpha = 1.0$
Maximum BP iterations	60
Post-processing	OSD with combination-sweep
OSD order	10
Reference decoder	Tesseract

Table 6.1: Baseline decoder and simulation settings.

6.1.2 Experimental Sequence

The early experiments first isolate the discrepancy and implicate the interface from BP to OSD. Message-dynamics experiments then show why an "oscillation causes logical failure" explanation is too narrow. Adaptive BP experiments identify the parity-check matrix representation as a more important degree of freedom. Finally, OSD and parity-check matrix interventions test whether useful information remains available despite a wrong BP hard decision and whether the failures depend on the particular Tanner graph representation.

Stage	Main experiments	Contribution to the final interpretation
Discrepancy isolation	Noise-model benchmark, BP schedule, forced OSD	The discrepancy appears in the tested circuit-level setting; BP schedule alone does not remove it.
Low-weight error diagnosis	Failure overlap, weight distribution	Excess BP+OSD failures include low-weight syndrome-consistent logical errors.
BP dynamics and stopping	Message dynamics, min-sum scaling, continued BP	Instability is primarily associated with difficulty reaching syndrome consistency; early stopping explains only a subset of logical failures.
Early stopping	Longer iterations	Early stopping on first syndrome-consistent correction is a real but non-universal mechanism.
BP-to-OSD information	Forced OSD, online selection	Useful BP belief may survive a wrong hard decision, but simple trajectory-based handoff is not robust end-to-end.
PCM representation	Equivalent PCM ensembles, same-PCM restarts, equivalent PCM transformations	Many disagreement cases are strongly representation dependent.
Redundant-check intervention	Adaptive redundant checks and equal-budget ablation	PCM diversity explains most of the repair gain; instability-based targeting provides only a modest additional benefit.

Table 6.2: Experimental stages and their contribution to the final interpretation.

6.2 Phenomenon Observation

6.2.1 Isolating the BP+OSD and Tesseract Discrepancy

The first benchmark progressively changes the noise model while keeping the code and decoders fixed, as shown in Figure 6.1. No visible discrepancy between BP+OSD and Tesseract is observed in the code-capacity or phenomenological noise at the available statistical resolution. The discrepancy first appears when circuit faults associated with the syndrome-extraction CNOTs are included and remains visible after simplifying the circuit to one syndrome-extraction round with perfect measurements and after removing idle and reset noise.

This process isolates a small circuit-level detector error model in which the effect can be studied without the additional complexity of long measurement histories. Although the restricted graph may retain much of the topology of the simpler Tanner graph, its variables and non-uniform priors are circuit-derived; the corresponding experiment is therefore analyzed as a circuit-level decoding problem.

The discrepancy also depends on the physical error rate. At comparatively high error rates, the two LER curves are not visibly distinct at the available statistical resolution. Below an empirical divergence point, denoted by p_{div}, BP+OSD develops additional failures, leading to a higher LER. This is a feature of the tested code, circuit, decoder parameters, and sampling resolution, rather than a code threshold.

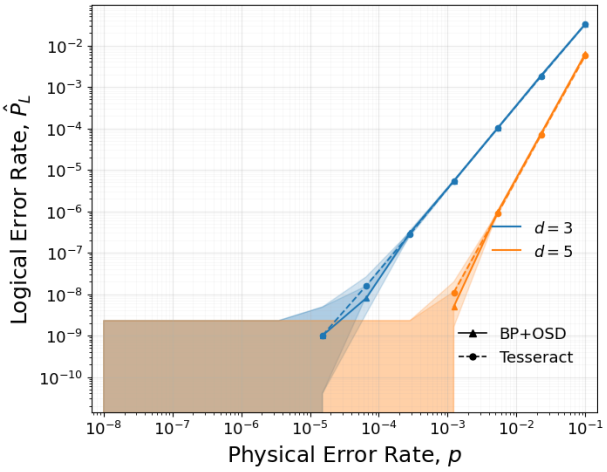
Setting	Observation	Interpretation
Code-capacity / phenomenological	No visible discrepancy	The discrepancy is not reproduced in simpler noise models.
Circuit-level CNOT faults	A clear low-p discrepancy appears	The effect is associated with circuit-level syndrome extraction.
One round, perfect measurement	The discrepancy persists	Long measurement histories and measurement flips are not required.
No idle / reset noise	The discrepancy remains visible	Idle and reset faults are not the primary source.

Table 6.3: Isolation of the discrepancy.

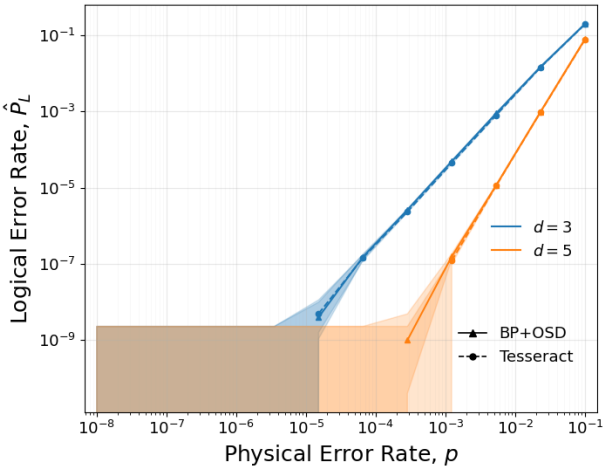
6.2.2 Controlled Decoder Diagnostics: CNOT Schedule, BP Schedule and Forced OSD

Three controlled modifications were introduced to distinguish between circuit-scheduling effects, BP message-scheduling effects, and the BP-to-OSD interface, as shown in Figure 6.2.

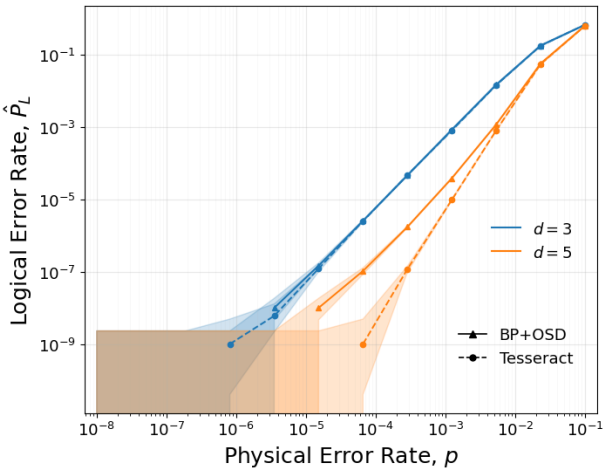
First, flooding BP was replaced by a serial schedule. Sequential updates change the order in which messages propagate and can suppress some oscillations. Although individual trajectories changed, the



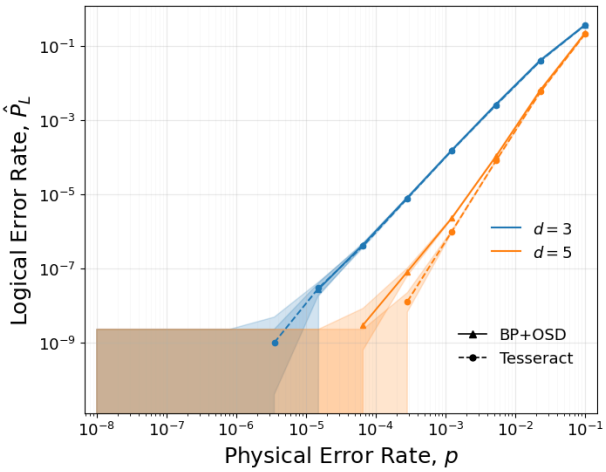
(a) Code-capacity



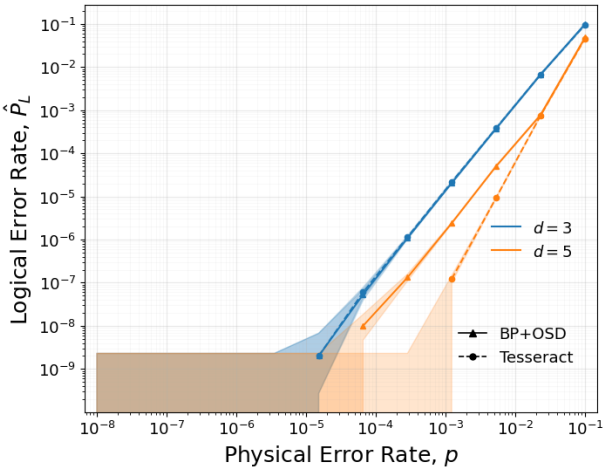
(b) 2-round phenomenological



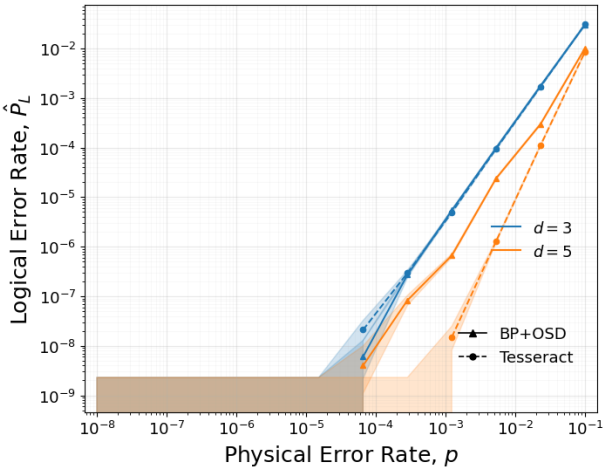
(c) 3-round circuit-level



(d) 1-round circuit-level with perfect measurement



(e) 1-round circuit-level with perfect measurement, no idle



(f) 1-round circuit-level with perfect measurement and reset, no idle

Figure 6.1: Benchmarks across code-capacity, phenomenological, and progressively simplified circuit-level noise models.

logical discrepancy was not systematically removed. Therefore, changing the message schedule alone is insufficient to explain the discrepancy.

Second, the ordering of CNOTs in the syndrome-extraction circuit was randomized instead of using the edge-coloring schedule. Changing the CNOT order modifies the propagation of circuit faults and hence the associated hook errors, while leaving the measured stabilizers unchanged. The discrepancy persisted across the tested orderings. This disfavors a particular CNOT ordering, and the hook errors induced by that ordering, as the primary explanation for the observed discrepancy.

Third, OSD was forced to run even when BP had already produced a syndrome-consistent hard decision. In the examined dominant low-weight errors and in the benchmark, this modification recovered the correct logical class in nearly all of the examined dominant low-weight cases and removed the visible LER discrepancy. The initial implication was therefore that BP can produce a syndrome-consistent but logically wrong estimate while still supplying belief useful enough for OSD.

Variant	Hypothesis tested	Experimental interpretation
Serial BP	Discrepancy mainly caused by synchronous flooding oscillations	Discrepancy persists; update order alone is insufficient.
Random CNOT ordering	Discrepancy is caused by the hook error associated with a particular CNOT schedule	Discrepancy persists across tested CNOT orderings; the specific circuit ordering is not the primary cause.
Forced OSD	OSD could repair a syndrome-consistent BP hard decision but is skipped	Dominant low-weight errors are recovered; early stopping of BP is implicated.

Table 6.4: Controlled decoder variants and their initial interpretation.

6.3 Failure Cases Overlap and Low-weight Errors

Error configurations are sampled and decoded independently using BP+OSD and Tesseract, and the overlap between their resulting failure sets is shown in Figure 6.3, which reveals a strong asymmetry. Almost every sampled Tesseract failure is also a BP+OSD failure, while BP+OSD contains an additional family of errors that is absent from the sampled Tesseract failure set. These excess errors generate the observed discrepancy at low physical error rates.

The sampled failures are further decomposed according to their error weight. As shown in Figure 6.4, the dominant low-weight BP failures already occur at weight 2. These samples frequently produce a syndrome-consistent BP hard decision, so OSD is not invoked. The weight-2 failures therefore expose the weakness of BP inference. At weight 3, the failure mechanism is different: OSD is invoked but does not return an estimate in the correct logical class. These cases are less directly attributable to BP+OSD alone and instead reflect the increased ambiguity of higher-weight error configurations (i.e., distinct error configurations belonging to different logical classes may have the same syndrome) together with the limitations of the OSD search.

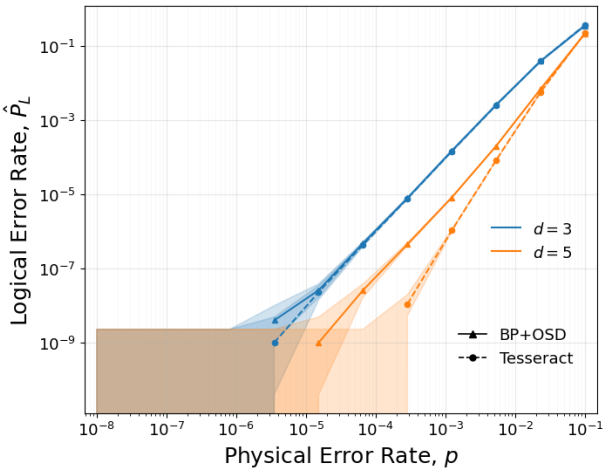
6.4 Message Dynamics: What Instability Measures and What It Does Not

The BP decoder records the check-to-variable messages and posterior LLRs at every iteration. For a check-to-variable edge $c \to v$, the one-step message residual is

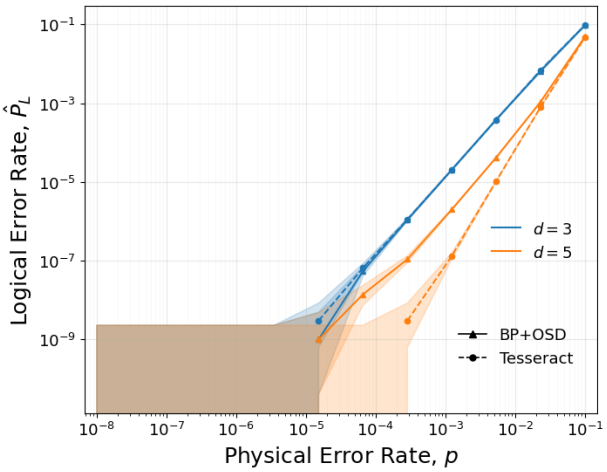
$$r_{c\to v}^{(t)} := \left| m_{c\to v}^{(t+1)} - m_{c\to v}^{(t)} \right|. \tag{6.1}$$

We construct several quantities based on this residual, including the maximum normalized late residual over all edges $R_{\max}$. The complete definitions are given in Appendix A.3.

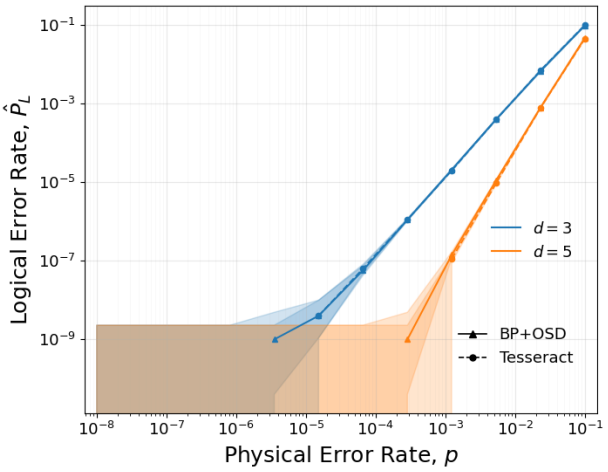
The quantitative result is a clear separation between syndrome consistency and logical correctness. $R_{\max}$ discriminates syndrome-inconsistent from syndrome-consistent BP hard decisions with AUROC



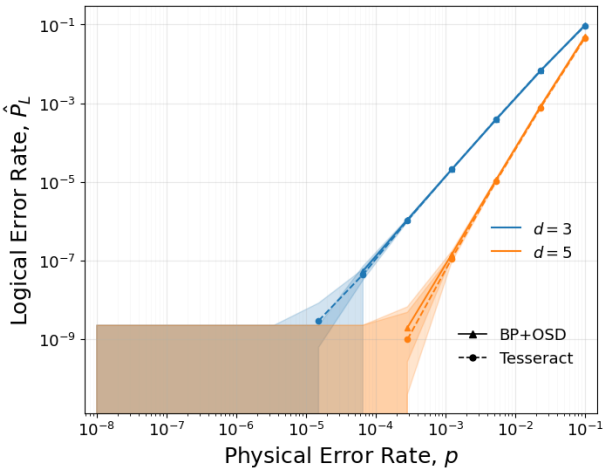
(a) 1-round circuit-level with perfect measurement, serial BP



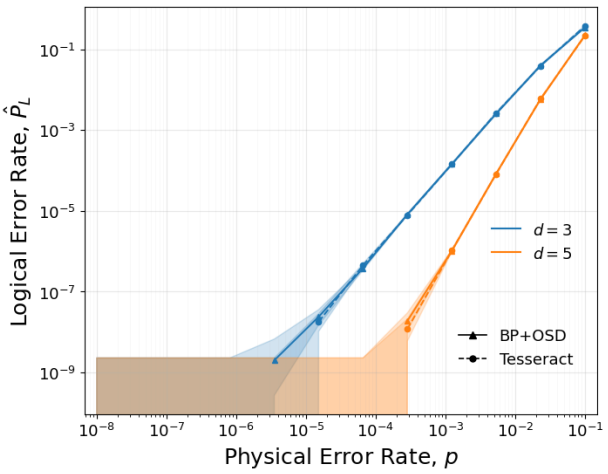
(b) 1-round circuit-level with perfect measurement and reset, no idle, random schedule



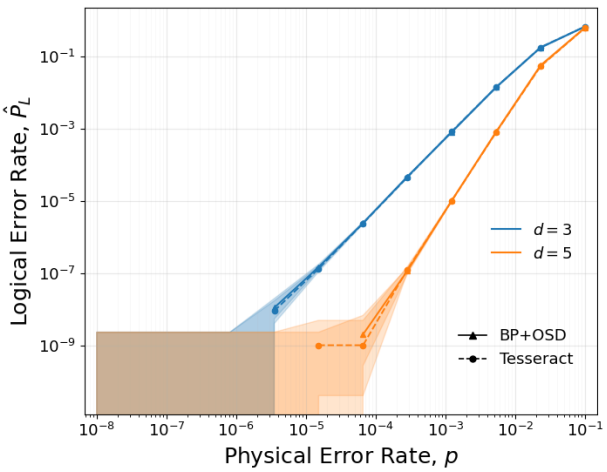
(c) 1-round circuit-level with perfect measurement, no idle, serial BP



(d) 1-round circuit-level with perfect measurement, no idle, random schedule, serial BP



(e) 1-round circuit-level noise with perfect measurement, forced OSD



(f) 3-round circuit-level noise, forced OSD

Figure 6.2: Benchmarks for serial BP, random CNOTs and forced OSD.

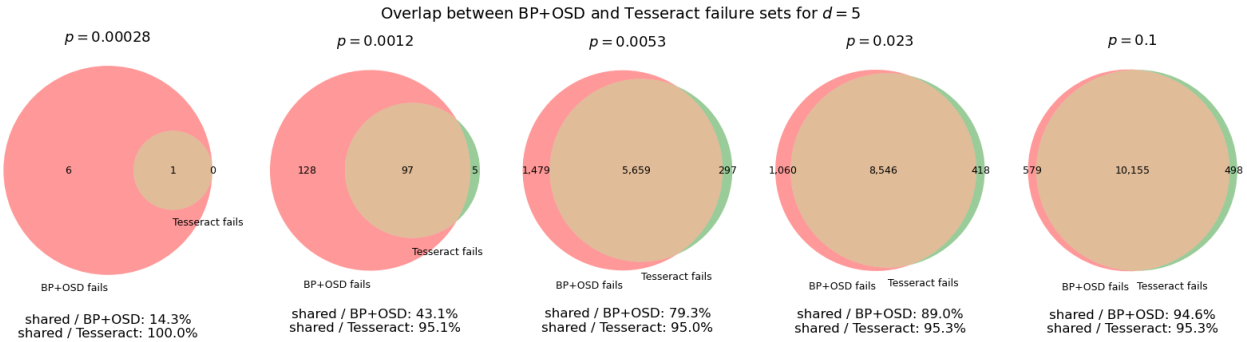


Figure 6.3: Overlap between BP+OSD and Tesseract failure sets for the distance-5 code at different physical error rates. Each panel shows the numbers of BP+OSD-only, Tesseract-only, and shared failures among the sampled error configurations. The overlap is strongly asymmetric: most Tesseract failures are shared with BP+OSD, while BP+OSD exhibits an additional family of failures, especially at low physical error rates.

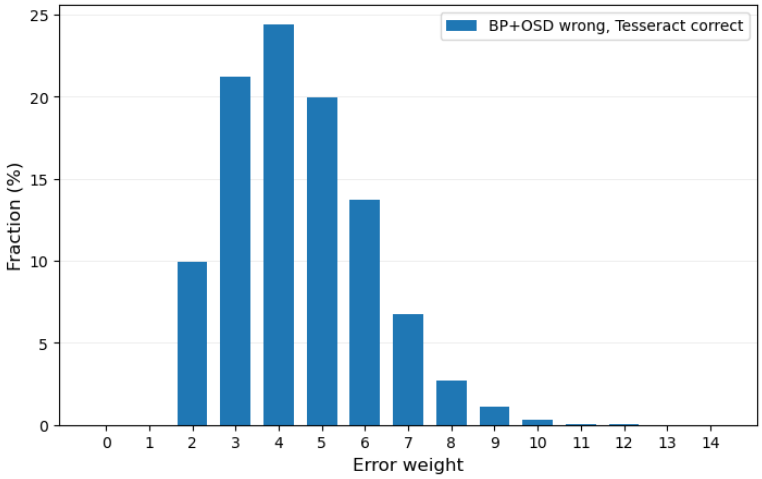


Figure 6.4: Error-weight distribution of the {BP+OSD wrong, Tesseract correct} samples for the distance-5 code. The distribution is normalized over these samples only. Low-weight failures already appear at weight 2.

0.910, but after conditioning on syndrome consistency it discriminates logically wrong from logically correct states with AUROC only 0.574. The same hierarchy appears across the dynamics quantities, as shown in Table 6.5.

Quantity	Inconsistent	Consistent	Consistent-correct	Consistent-wrong
$R_{\max}$	0.851	0.158	0.153	0.236
R_{mean}	0.351	0.057	0.055	0.093
$F_{\max}$	0.640	0.213	0.209	0.281
$P^{(2)}_{\max}$	2.866	0.950	0.943	1.071
$\overline{I}^{\text{msg}}$	2.191	0.356	0.344	0.552
$\overline{I}^{\text{belief}}$	3.306	0.550	0.533	0.832
$\overline{C}$	0.226	0.092	0.089	0.142

Table 6.5: Message-dynamics statistics for syndrome consistency and logical correctness.

The second important result is that logical failure survives even after BP convergence. At the strict threshold $R_{\max} < 10^{-8}$, 57.9% of syndrome-consistent logical-wrong cases are already classified as stable, compared with 77.7% of syndrome-consistent logical-correct cases. The logical-wrong stable fraction is unchanged through thresholds up to 10^{-4}. Thus a trajectory can satisfy

$$R_{\max} \simeq 0, \qquad H\hat{e} = s, \qquad L\hat{e} \neq \ell_{\text{true}}, \tag{6.2}$$

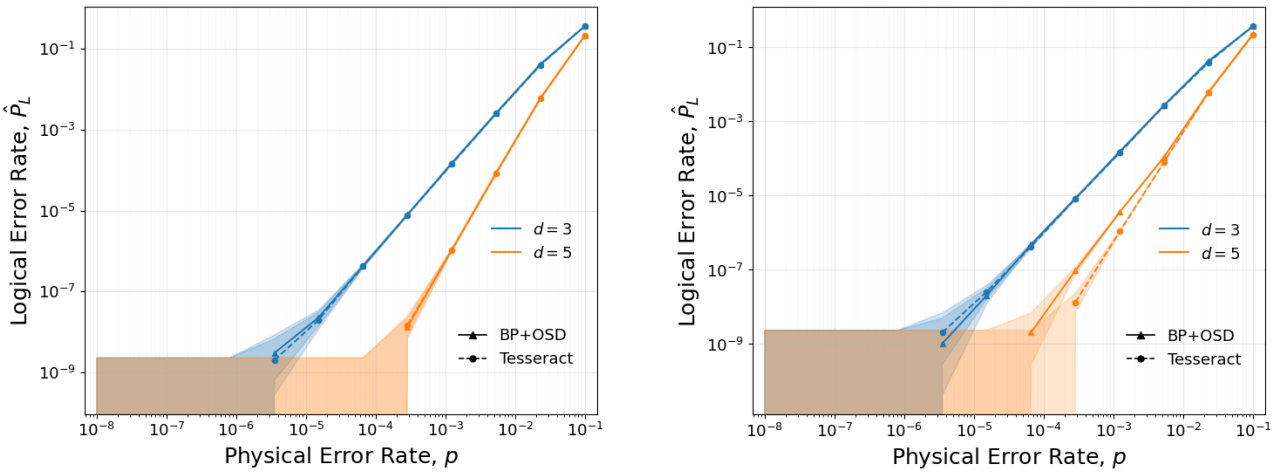
which directly rules out persistent oscillation as a universal explanation for logical error.

6.4.1 Min-sum Scaling and the Limits of Oscillation Suppression

The min-sum scaling factor α provides a way to control the strength of repeated messages:

$$m^{(t)}_{c\to v} = \alpha \left((-1)^{s_c} \prod_{v' \in \mathcal{N}(c)\setminus\{v\}} \operatorname{sign} m^{(t-1)}_{v'\to c} \right) \min_{v' \in \mathcal{N}(c)\setminus\{v\}} \left| m^{(t-1)}_{v'\to c} \right|. \tag{6.3}$$

Attenuating the messages with a smaller α empirically shifts the divergence point p_{div} to lower physical error rates, whereas amplifying the messages shifts it to higher rates, as shown in Figure 6.5. In the examined samples, attenuation is also associated with fewer or weaker belief oscillations and closer agreement with Tesseract.



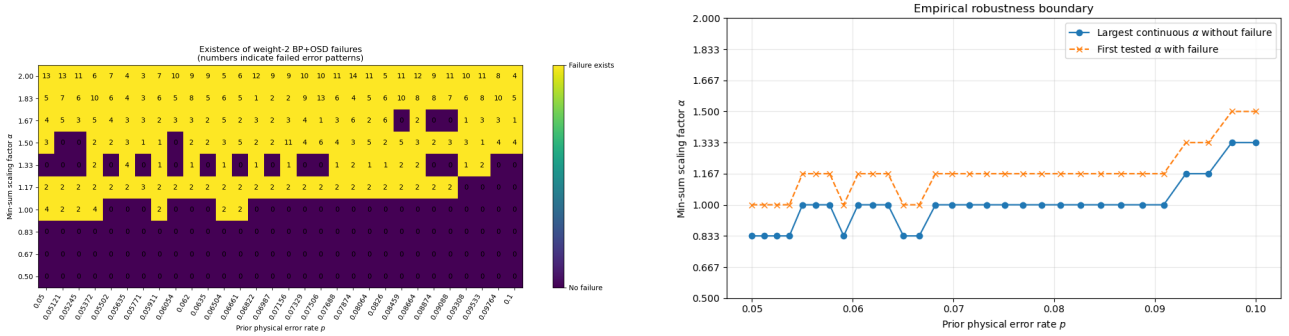
(a) 1-round Circuit-level with Perfect Measurement, $\alpha = 0.8$

(b) 1-round Circuit-level with Perfect Measurement, $\alpha = 1.2$

Figure 6.5: Benchmark on Scaling Factor

Figure 6.6 shows a broad failure-free region at small scaling factors: no weight-2 failures are observed for $\alpha \leq 0.83$ over the tested range of p, whereas failures become prevalent for $\alpha \geq 1.5$. The intermediate region is not perfectly monotonic because individual error patterns can appear or

disappear as the priors vary. The right figure reports a conservative continuous boundary rather than fitting every isolated cell. Its upward trend indicates that the largest failure-free scaling factor increases with p over this finite grid.



(a) Number of failing weight-2 errors over the tested (p, α) grid. Dark cells indicate no observed failure.

(b) Failure-free scaling envelope extracted from the discrete grid in (a).

Figure 6.6: Dependence of weight-2 errors on the prior physical error rate p and min-sum scaling factor α in BP+OSD. (a) The cell annotations give the number of failing errors. (b) The blue curve is the largest continuously failure-free tested value of α, while the orange curve is the first tested value producing a failure. The boundary is empirical and specific to the sampled parameter grid.

This behaviour is consistent with the combined effect of cycles and overly strong repeated messages. The evidence remains empirical: changing α modifies both the approximate fixed points and the dynamics, so it should not be claimed that a smaller α universally yields exact posterior probabilities. The extracted curve is not a decoder threshold; such a claim would require several code sizes and finite-size scaling.

6.4.2 Early Stopping: a Real but not Universal Mechanism

After deduplication by $(p, s, \ell_{\text{true}})$, we obtain 6 unique weight-2 instances for which the benchmark BP output is syndrome-consistent but logically wrong. These instances provide a direct stopping-rule diagnostic, as shown in Figure 6.7. BP stops as soon as the first syndrome-consistent hard decision is obtained, so any later evolution of the BP trajectory is discarded. In all 6 instances, the first syndrome-consistent state lies in the wrong logical class. When early stopping is disabled, 5 trajectories later reach a syndrome-consistent and logically correct state, while the remaining one cannot reach a logically correct state within 100 iterations.

This establishes a mechanism: early stopping at the first syndrome-consistent state prevents BP from reaching the correct logical class. However, the remaining instance implies that simply allowing additional iterations does not resolve all logical failures.

Observation	Key result	Interpretation
Residual predicts syndrome inconsistency	AUROC = 0.910	Instability is a strong marker of constraint-satisfaction difficulty.
Residual predicts logical failure after consistency	AUROC = 0.574	Once the syndrome is satisfied, instability carries little information about the logical class.
Stable consistent logical failures	57.9%	Convergence and syndrome consistency do not guarantee logical success.
Weight-2 errors repaired by continued BP	5/6	Early stopping can freeze a wrong logical class.

Table 6.6: Summary of the first message-dynamics and early-stopping study.

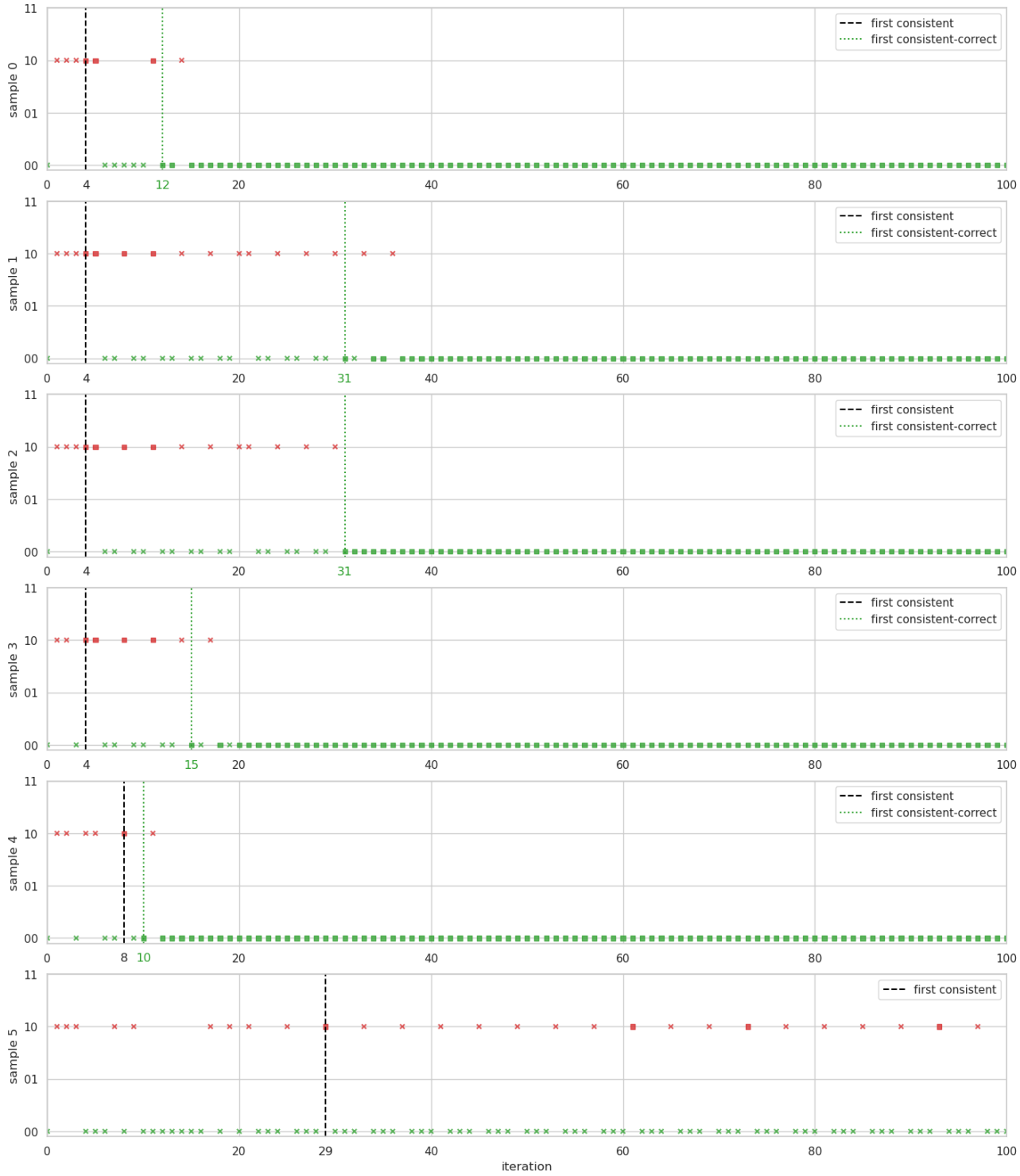


Figure 6.7: BP trajectories for all 6 unique weight-2 failures. The black dashed line indicates the first iteration where BP produces a syndrome-consistent estimate, while the green dashed line indicates an estimate that is both syndrome-consistent and logically correct. Squares denote syndrome-consistent estimates, whereas crosses denote inconsistent ones; green denotes a logically correct estimate, and red denotes a logically incorrect one. In every case, the first syndrome-consistent correction lies in the wrong logical class. For 5 of the 6 inputs, continued iterations subsequently produce a syndrome-consistent correction in the correct logical class.

6.5 Taxonomy of BP Failures

To determine how representative the previously identified mechanisms are, we perform a larger follow-up study of the weight-2 and weight-3 instances of the distance-5 code. For each failed instance, we examine both the state at the maximum iteration and the state obtained by continuing BP beyond that iteration. This yields four mutually exclusive failure classes:

1. **Dynamic syndrome failure** ($\forall t, H\hat{e}^{(t)} \neq s; L\hat{e}^{(t_{\text{stop}})} \neq \ell_{\text{true}}$): the state at the maximum iteration is syndrome-inconsistent.
2. **Premature first-consistent failure** ($H\hat{e}^{(t_{\text{stop}})} = s, L\hat{e}^{(t_{\text{stop}})} \neq \ell_{\text{true}}; \exists t > t_{\text{stop}}, H\hat{e}^{(t)} = s, L\hat{e}^{(t)} = \ell_{\text{true}}$): the first consistent state is logically wrong, but a later continued iteration becomes consistent and correct.
3. **Stable persistent logical failure** ($H\hat{e}^{(t_{\text{stop}})} = s; \forall t, L\hat{e}^{(t)} \neq \ell_{\text{true}}; R_{\max} < \tau$): the output is consistent and wrong, no later correct state is seen, and the late residual is below the threshold.
4. **Unstable persistent logical failure** ($H\hat{e}^{(t_{\text{stop}})} = s; \forall t, L\hat{e}^{(t)} \neq \ell_{\text{true}}; R_{\max} \geq \tau$): the output is consistent and wrong, no later correct state is seen, and the late residual remains above the threshold.

	Weight-2		Weight-3	
Failure type	Cases	Fraction	Cases	Fraction
Dynamic syndrome failure	256	95.52%	7,832	86.76%
Premature first-consistent failure	10	3.73%	97	1.07%
Stable persistent logical failure	0	0.00%	622	6.89%
Unstable persistent logical failure	2	0.75%	476	5.27%
Total	268	100%	9,027	100%

Table 6.7: Exclusive baseline failure classes separated by error weight. Fractions are normalized independently within each weight. The threshold τ is 10^{-8}.

Unless otherwise specified, all *baseline failures* mentioned in the following experiments refer to the weight-2 and weight-3 failures discussed here.

6.6 From BP to OSD

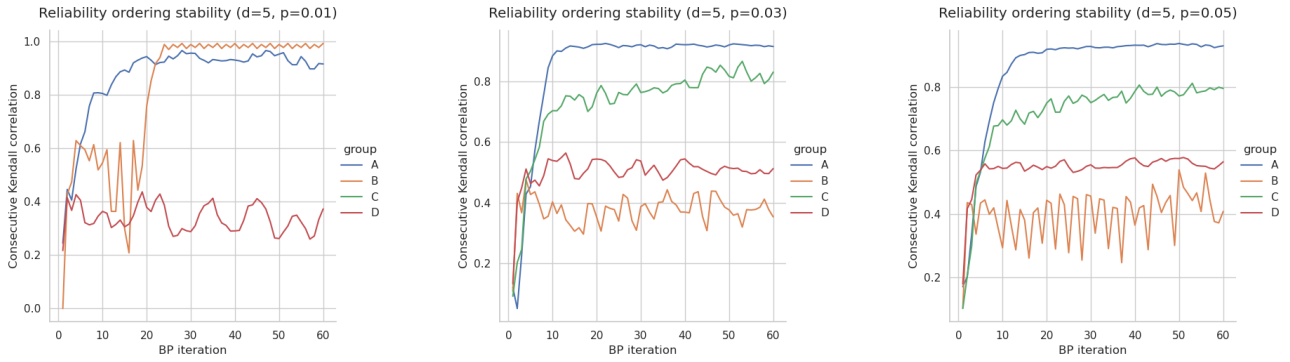
6.6.1 BP Beliefs and OSD Reliability

The failure of the BP hard decision does not necessarily imply that the underlying BP beliefs are uninformative. To separate these two effects, we run BP for a fixed number of iterations and compare its hard decision with OSD corrections constructed from the same posterior beliefs. In a small experiment containing 200 samples drawn from the weight-2 and weight-3 error instances, with $\alpha = 0.7$ and 10 iterations, the BP hard decision is logically correct on only 44.0% of the tested samples, whereas OSD-0 and OSD-1 using the same BP beliefs succeed on 79.5% and 88.0%, respectively. Thus, BP may fail to identify the correct estimate directly while still providing reliability information that allows OSD to recover the correct logical class.

We then examine how the useful beliefs evolve along the BP trajectory. At each iteration, OSD is reapplied using the current posterior reliability ordering, allowing us to identify when the BP beliefs become informative enough to give a correct correction. To determine whether this information is persistent or only transient, we compare reliability rankings across iterations. Then, we test several instability-aware modifications of the BP reliability to determine whether message dynamics can further improve OSD performance. The related modifications are summarized in Appendix A.4.

Group	Condition	Meaning
A	standard BP+OSD correct, Forced OSD correct	Control group in which both standard BP+OSD and forced OSD succeed.
B	standard BP+OSD wrong, Forced OSD correct	The reliability provided by BP still contains useful information, but the standard BP+OSD fails; this is the primary group of interest studied.
C	standard BP+OSD wrong, Forced OSD wrong	Difficult control group in which decoding still fails even when forced OSD is applied.
D	plain BP syndrome-inconsistent	Samples for which the BP hard decision does not satisfy $H\hat{e} = s$.

Table 6.8: Analysis groups used in the BP reliability trajectory experiments. Groups A–C are defined by standard and forced OSD outcomes, while Group D identifies syndrome-inconsistent plain BP outputs.



(a) $p = 0.01$ (b) $p = 0.03$ (c) $p = 0.05$

Figure 6.8: Consecutive reliability ordering Kendall correlation in different groups. Kendall correlation is computed between the reliability orderings at iterations t and $t + 1$, and averaged within the same group at each iteration. Successful trajectories (Group A) develop a highly stable ordering, while syndrome-inconsistent trajectories (Group D) remain persistently unstable. The recoverable-failure trajectories (Group B) are strongly prior-dependent: their rankings become nearly stable at $p = 0.01$, but remain substantially more unstable for $p = 0.03$ and $p = 0.05$.

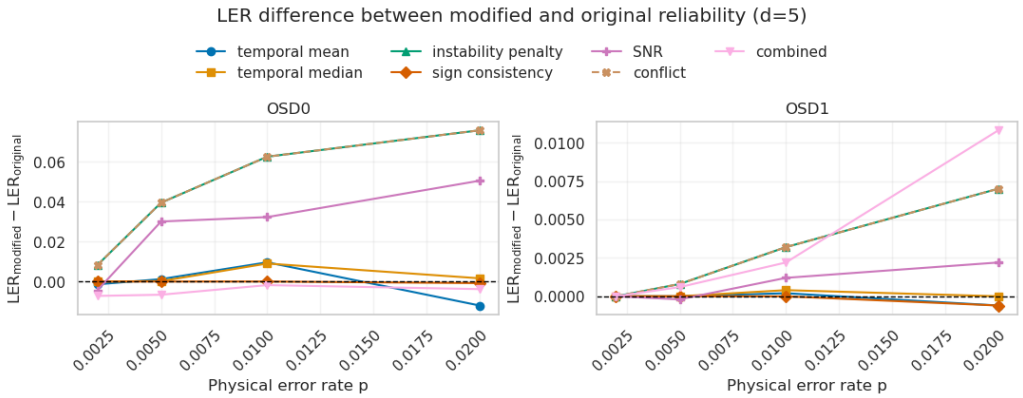


Figure 6.9: LER Difference between dynamics-aware reliability modifications and the original final BP reliability for the $d = 5$ code, shown for OSD-0 (left) and OSD-1 (right). Most tested modifications do not consistently outperform the original one. The result suggests that although BP dynamics affect the reliability ordering, a simple scalar penalty based on these dynamics is not sufficient to improve OSD.

The trajectory analysis shows that successful trajectories (Group A) generally develop a stable reli-

ability ordering, whereas syndrome-inconsistent trajectories (Group D) remain substantially unstable. The recoverable-failure trajectories (Group B) exhibit an intermediate and strongly prior-dependent behaviour, indicating that reliability stability is associated with decoding success but is not by itself sufficient to characterize it. Moreover, the tested dynamics-aware modifications do not always improve the original reliability. Thus, BP dynamics provide useful diagnostic information about the evolution of the reliability ordering, but simple instability-based penalties are not sufficient to improve the reliability supplied to OSD.

6.6.2 Reliability Windows and Online Selection

We next ask whether the endpoint BP reliability is itself the best information available to OSD, or whether more informative reliability can appear earlier in the trajectory. Let $\mathcal{G}$ contain samples for which standard BP+OSD fails, while forced OSD and Tesseract both succeed. Let $\mathcal{C}$ contain samples for which both standard and forced OSD fail, while Tesseract succeeds. OSD-0 and OSD-1 are then evaluated separately along the same BP trajectories.

First we perform a best-snapshot analysis. OSD is applied at each iteration using the reliability available at that iteration, whose result is called a snapshot. A sample is counted as successful if any iteration up to the endpoint yields an estimate in the correct logical class. This analysis is retrospective and does not define a practical stopping rule. Instead, it measures whether a more informative reliability window exists somewhere along the trajectory.

The difference between endpoint and best-snapshot performance is substantial. For $\mathcal{G}$ at $p = 0.05$, endpoint OSD-0 and OSD-1 succeed on 13.3% and 88.3% of the samples, whereas the best-snapshot success rates reach 95.3% and 100%, respectively. For $\mathcal{C}$, despite the failure of endpoint Forced OSD by construction, successful OSD iterations are still frequently present earlier in the trajectory; at $p = 0.05$, best-snapshot OSD-1 succeeds on 96.0% of the samples.

Group	p	Endpoint OSD-0	Endpoint OSD-1	Best-snapshot OSD-0	Best-snapshot OSD-1
$\mathcal{G}$	0.03	24.4%	95.1%	97.6%	100.0%
$\mathcal{C}$	0.03	25.3%	2.7%	97.3%	97.3%
$\mathcal{G}$	0.05	13.3%	88.3%	95.3%	100.0%
$\mathcal{C}$	0.05	17.3%	0.7%	98.0%	96.0%

Table 6.9: Endpoint versus best-snapshot OSD success rate in baseline failures.

These results show that useful reliability information can be transient. A BP trajectory may pass through an intermediate reliability that enables successful OSD even when the endpoint reliability does not. The failure is therefore not always caused by an absence of useful beliefs, but may instead arise because the OSD takes beliefs at an unfavorable point in the trajectory.

The remaining question is whether such favorable iterations can be identified without retrospective knowledge. To test this, we construct two snapshot-selection strategies. The *online handoff* strategy tracks the BP trajectory and invokes OSD-1 at the first iteration whose predicted snapshot quality exceeds a prescribed threshold. The *prefix-buffer* strategy instead stores the reliability snapshots observed up to the standard BP endpoint and applies OSD-1 to the previously observed snapshot with the highest predicted snapshot quality. Both methods use only information available up to the decision time; their detailed construction is given in Appendix A.4.1.

On the $p = 0.02$ benchmark based on the cases where BP+OSD fails but Tesseract succeeds, the online handoff strategy repairs approximately 51.5% of the cases, while the prefix-buffer strategy increases this repair rate to 74.3%. However, the same interventions also convert many previously successful cases into logical failures, leading to a worse overall logical error rate.

The main difficulty is therefore not the existence of useful intermediate reliability, but its online selection. BP trajectories often contain enough information for successful OSD before the endpoint, yet the tested selection rules do not identify these favorable iterations safely enough to improve end-to-end decoding performance.

6.7 Mechanism Attribution and Parity-check Matrix Dependence

We now return to the original mechanism question: why does BP+OSD fail on instances that Tesseract decodes correctly? We focus on the disagreement set $\mathcal{G}_D$ containing the instances on which BP+OSD fails but Tesseract succeeds.

To distinguish between possible sources of these failures, we apply several interventions to the same instances while keeping the syndrome and priors fixed. The interventions test different hypotheses: (i) sum-product BP tests dependence on the BP update approximation; (ii) repeated runs on the same parity-check matrix test whether additional updates on an unchanged Tanner graph are sufficient; (iii) equivalent parity-check matrices test sensitivity to the Tanner graph representation itself; (iv) low-order OSD tests how much of the failure can be repaired by post-processing the BP reliability.

For each instance, an ensemble of M mathematically equivalent parity-check matrices is constructed and BP is run independently on each corresponding Tanner graph. For this mechanism test, an instance is counted as recovered if at least one run produces an estimate in the correct logical class. The resulting conditional success rates on $\mathcal{G}_D$ are shown in Table 6.10. The strongest BP-side effect is obtained by changing the Tanner graph representation.

Intervention	$M = 2$	$M = 4$	$M = 8$	$M = 16$
Random equivalent parity-check matrix ensemble	21.8%	33.9%	45.9%	59.2%
Same parity-check matrix restart ensemble	6.6%	9.5%	12.7%	17.1%
Sum-product (single)		10.4%		
Low-order OSD (single)		16.1%		

Table 6.10: Conditional recovery rates on the disagreement set $\mathcal{G}_D$. For the random equivalent parity-check matrix ensemble, M denotes the number of mathematically equivalent parity-check matrices tested; for the same parity-check matrix restart ensemble, M denotes the number of repeated BP runs on the unchanged parity-check matrix, but each time with a small random perturbation applied to the inital messages. For these two ensemble interventions, a sample is counted as recovered if at least one of the M runs returns an estimate in the correct logical class. The sum-product and low-order OSD entries are single-intervention diagnostics and are included for comparison rather than M decoders.

Equivalent parity-check matrices are substantially more effective than repeated runs on the same parity-check matrix. At $M = 16$, changing the representation repairs 59.2% of the disagreement cases, compared with 17.1% for same parity-check matrix restarts. These comparisons suggest that the failures cannot be explained primarily by too few repeated runs. Instead, they point to a strong dependence of BP on the Tanner graph representation used to express the same decoding constraints.

To test this representation dependence more directly, we consider two classes of mathematically equivalent parity-check matrix transformations:

- **Basis replacement**, in which the parity-check basis is changed while preserving the same row space,
$$H' = BH, \qquad s' = Bs, \tag{6.4}$$
This transformation leaves the constraint space unchanged without increasing the number of checks.

- **Redundant addition**, in which linear combinations of the original parity checks are appended as additional rows,
$$H' = \begin{bmatrix} H \\ RH \end{bmatrix}, \qquad s' = \begin{bmatrix} s \\ Rs \end{bmatrix}. \tag{6.5}$$
These additional rows do not change the error mechanisms, but introduce extra check nodes and message-passing pathways in the Tanner graph.

As shown in Table 6.11, basis replacement alone already recovers a large fraction of the disagreement cases, showing that BP is sensitive to the particular representation of the same constraint space.

Redundant addition is consistently more effective. This indicates that, beyond representation dependence itself, the additional redundant checks and message-passing pathways provide a further benefit.

p	At least one correct		Single success	
	Basis	Redundant	Basis	Redundant
0.005	100.0%	100.0%	26.7%	35.3%
0.010	81.8%	90.9%	22.2%	27.9%
0.020	87.1%	89.1%	17.4%	27.8%
0.030	85.6%	89.4%	17.2%	30.3%
0.050	69.0%	87.3%	15.1%	28.1%
Total	79.1%	88.7%	16.8%	28.7%

Table 6.11: Comparison of basis replacement and redundant addition transformations on $\mathcal{G}_D$. "At least one correct" denotes the fraction of cases for which at least one representation gives the correct logical class, while "single success" denotes the average success rate of an individual transformed representation.

Taken together, these experiments identify parity-check matrix representation dependence as a major contributor to the discrepancy between BP+OSD and Tesseract. For many disagreement instances, the information needed to identify the correct logical class is not absent from the decoding problem. Instead, BP's ability to extract that information depends strongly on how the same constraints are factorized into a Tanner graph. This observation motivates the Adaptive BP experiments below, which ask whether useful alternative parity-check matrices can be generated deliberately rather than found by randomly sampling alternative representations.

6.8 Representation-Based Interventions

The previous experiments show that many BP failures are sensitive to the parity-check matrix representation. This motivates us to design a decoder that deliberately constructs alternative parity-check matrices rather than repeatedly running BP on the original parity-check matrix.

6.8.1 Adaptive BP

A detailed description is given in Appendix A.5. Adaptive BP begins by running BP on the original parity-check matrix H and identifying variables associated with large message or belief instability. Redundant parity checks are then constructed as low-weight linear combinations of existing rows, with preference given to checks that involve these unstable variables. A small collection of augmented matrices

$$H^{(1)}, H^{(2)}, \ldots, H^{(M)}$$

is obtained by appending selected redundant checks to the original parity-check matrix. BP is run independently on these equivalent representations, and the resulting candidates are evaluated using the same syndrome constraints and channel priors. The added rows do not change the feasible error set; they only modify the factorization and message-passing pathways seen by BP. Thus, Adaptive BP combines two ingredients: *parity-check matrix diversity*, obtained by introducing equivalent Tanner graph representations, and an *instability-based heuristic*, used to decide which redundant checks to add. The experiments below separate the contribution of these two effects.

6.8.2 Adaptive BP Repair is Strongly Mechanism Dependent

We first ask whether Adaptive BP can repair all BP failures. The results in Table 6.12 show a strong dependence on the BP failure taxonomy.

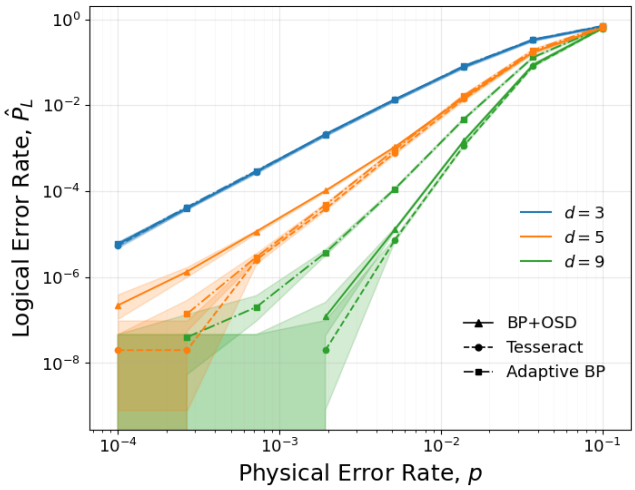


Figure 6.10: Benchmark of Adaptive BP for the $d = 3, 5, 9$ codes in 3-round circuit-level noise. Adaptive BP performs better than BP+OSD for $d = 3$ and $d = 5$, but worse for $d = 9$.

	Weight-2		Weight-3	
BP failure type	Logical-correct	Syndrome-consistent	Logical-correct	Syndrome-consistent
Dynamic syndrome failure	99.6%	100.0%	85.4%	95.1%
Premature first-consistent failure	100.0%	100.0%	80.4%	100.0%
Stable persistent logical failure	–	–	0.0%	100.0%
Unstable persistent logical failure	100.0%	100.0%	12.8%	100.0%

(a) Adaptive BP on baseline failures.

BP+OSD failures	Logical-correct	Syndrome-consistent
1552	524 (33.76%)	814 (52.45%)

(b) Adaptive BP on weight-4 BP+OSD failures of the distance-9 code.

Table 6.12: Adaptive BP performance on selected failure samples. For the distance-5 code, weight-2 and weight-3 BP failures are separated according to the BP failure taxonomy. Adaptive BP produces syndrome-consistent estimates for nearly all cases and successfully repairs almost all weight-2 failures. For the distance-9 code, weight-4 samples are from BP+OSD failures. Adaptive BP repairs 33.76% of these failures and produces syndrome-consistent corrections for 52.45%.

For the distance-5 code, Adaptive BP is highly effective on low-weight failures whose BP trajectories remain dynamically unresolved. Almost all weight-2 dynamic syndrome failures, premature first-consistent failures, and unstable persistent logical failures are repaired, with the resulting candidates satisfying the syndrome constraints. The corresponding logical-correct rates decrease for weight-3 errors, indicating that the benefit of changing the Tanner graph representation becomes less reliable as the error configuration becomes more difficult.

Stable persistent logical failures behave differently. Adaptive BP produces syndrome-consistent candidates for all of these cases, yet none are moved into the correct logical class. This shows that modifying the parity-check matrix representation can effectively preserve syndrome consistency but has difficulty resolving logical-class ambiguity.

The same limitation becomes more apparent at larger distance. On the distance-9 weight-4 BP+OSD failures, Adaptive BP makes 52.45% of the instances syndrome-consistent but repairs only 33.76% logically. Thus, the representation-based intervention remains useful beyond the small distance-5 examples, but its effectiveness is substantially reduced on the more difficult failures. Overall, Adaptive BP is most effective on failures associated with unstable BP dynamics, whereas stable logical-wrong cases are largely insensitive to the tested representation changes.

6.8.3 Ablation: Parity-check Matrix Diversity

The previous experiment does not yet determine whether Adaptive BP succeeds because of the modified representation, instability heuristics or simply because it performs more message updates. To separate these effects, we compare several controls under approximately matched edge-update budgets,

$$B_{\text{edge}} = \sum_j T_j E_j, \tag{6.6}$$

where T_j is the number of BP iterations performed on view j and E_j is the number of Tanner graph edges in that view. Longer BP spends the same budget on the original parity-check matrix. Restarted BP distributes the budget across repeated runs on the same parity-check matrix. The random redundant-view control instead constructs augmented parity-check matrices with the same number and approximate row-weight distribution of redundant checks as Adaptive BP, but selects them without using the instability heuristics.

Variant	Logical-correct	Syndrome-consistent	Budget ratio to Adaptive BP
Adaptive BP	74.21%	95.58%	1.000
Longer BP	3.03%	14.16%	0.998
Restarted BP	0.40%	9.93%	0.998
Random redundant-view BP	73.16%	94.27%	1.048

Table 6.13: Equal-budget ablation over all weight-2 and weight-3 failures on the distance-5 code.

The large gap between Adaptive BP and the longer or restarted BP controls shows that the observed gain cannot be explained primarily by additional message updates. In contrast, the random redundant-view control reaches a repair rate of 73.16%, only 1.05 percentage points below Adaptive BP. Under this tested configuration, most of the gain therefore comes from exposing BP to alternative parity-check matrix representations, while the instability-based selection rule provides only a modest additional improvement. This ablation refines the interpretation of Adaptive BP: its main advantage is not the identification of a uniquely problematic unstable region, but the generation of alternative Tanner graph representations on which BP can follow different message-passing trajectories. The instability heuristic acts mainly as a weak bias over this broader representation-diversity effect.

6.9 Experiment Summary

Taken together, the experiments support the hierarchy

$$\text{BP dynamics} \longrightarrow \text{syndrome resolution} \longrightarrow \text{logical class discrimination}.$$

The stages correspond to different failure modes and, importantly, respond to different interventions:

Dynamical syndrome failure. Persistent message instability is strongly associated with failure to satisfy the syndrome. Adaptive or random redundant-view BP repairs this class at high rate, whereas equal-budget extra iterations and restarts on the original PCM do not. The demonstrated intervention is therefore graph diversification rather than waiting longer.

Premature first-consistent failure. A first syndrome-consistent state can be logically wrong while a later state is correct. This is directly observed in weight-2 and weight-3 errors in $d = 5$ code. Stopping should therefore be treated as an independent decoder component.

Stable persistent logical failure. A substantial set of BP trajectories are numerically stable and syndrome-consistent but logically wrong. Adaptive BP repairs none of the stable persistent logical failures. This class requires an explicit mechanism for discriminating logical classes, not simply additional checks.

Unstable persistent logical failure. Some trajectories remain dynamically unstable while already producing syndrome-consistent but logically wrong estimates. Redundant-check augmentation repairs only a minority of these cases, substantially less than for dynamical syndrome failures. This indicates that instability alone is not sufficient to determine whether graph modification can recover the correct logical class.

7 Conclusion

This work investigated why belief propagation with ordered-statistics decoding (BP+OSD) performs worse than a search-based decoder (Tesseract). The main distinction is between syndrome consistency, BP message stability, and correctness of the logical class. In quantum decoding, these properties are not equivalent: one syndrome class contains several logical cosets, so the syndrome constraint $H\hat{e} = s$ is not by itself a certificate of successful correction.

The experiments compared BP+OSD with Tesseract on a small triangular code of distance 5. No visible discrepancy was observed under code-capacity or phenomenological noise. A separation appeared after circuit faults on the syndrome-extraction CNOT gates were included and remained in a restricted one-round setting with perfect measurements and without idle or reset noise. At low physical error rates, the additional BP+OSD failures were dominated by weight-2 error configurations. BP could return a syndrome-consistent estimate in the wrong logical class, after which the standard stopping rule terminated before OSD was invoked. Forcing OSD to run recovered the correct logical class in the examined dominant failures and closed the visible discrepancy in the tested minimal benchmark. This identifies premature acceptance of the BP output, rather than an insufficient OSD search space, as the principal mechanism in these cases.

Serial BP changed individual trajectories but did not systematically eliminate the discrepancy. The belief histories instead showed that message instability was localized: a small set of variables displayed repeated sign changes, large LLR variations, or weak final confidence, while most variables remained stable. Reducing the min-sum scaling factor weakened the observed instability and shifted the empirical divergence point toward lower error rates. These observations are consistent with short cycles and repeatedly amplified messages contributing to overconfident BP decisions, although the outcome also depends on the syndrome and the non-uniform circuit priors.

The experiments reveal a hierarchy of BP failure mechanisms. Message instability is primarily associated with failures of syndrome resolution, whereas logical-class errors can persist even after numerical convergence and syndrome satisfaction. BP failures are also strongly dependent on the parity-check representation: equivalent Tanner graph representations repair substantially more failures than additional iterations or restarts on the original graph. Equal-budget ablations show that most of the improvement achieved by Adaptive BP arises from this representation diversity rather than from the instability-based selection heuristic. Representation changes are highly effective for dynamically unresolved failures but do not resolve stable logical-class ambiguity.

The conclusions are limited to the small code instances, restricted circuits, and decoder configurations studied here. The observed divergence point is a finite-size feature rather than an asymptotic threshold. Future work should test multiple code distances and qLDPC families under full circuit-level noise, calibrate instability and logical-confidence criteria against accurate logical posteriors, and compare the proposed refinements under equal computational budgets. The central conclusion is that syndrome satisfaction should be treated as a constraint rather than a success certificate. Monitoring logical ambiguity and BP stability offers a route to preserving the efficiency of BP while reducing its most consequential low-weight failures.

A Supplementary Material

A.1 Tesseract Decoder

Tesseract is a search-based *most-likely-error* (MLE) decoder for detector error models. Its objective is to identify the most probable individual subset of elementary error mechanisms that reproduces the observed detection events. It therefore solves a non-degenerate MLE problem: probabilities are not summed over all error subsets with the same logical action [3].

Let $\mathcal{E}$ be the set of elementary error mechanisms and $\mathcal{D}$ the set of detectors. An error $e \in \mathcal{E}$ occurs independently with probability $p(e) \in (0, 1/2]$ and activates a detector set $D(e) \subseteq \mathcal{D}$. For a subset $F \subseteq \mathcal{E}$, its detector signature is the symmetric difference

$$D(F) := \bigoplus_{e \in F} D(e). \tag{A.1}$$

If $\mathcal{S} \subseteq \mathcal{D}$ is the observed set of detection events, then

$$\Pr(F) = \prod_{e \in F} p(e) \prod_{e \in \mathcal{E} \setminus F} \big(1 - p(e)\big). \tag{A.2}$$

After removing a term independent of F, maximizing this probability is equivalent to assigning each error the non-negative cost

$$w(e) := -\log \frac{p(e)}{1 - p(e)} = \log \frac{1 - p(e)}{p(e)}, \qquad w(F) := \sum_{e \in F} w(e), \tag{A.3}$$

and solving

$$F^* := \operatorname*{arg\,max}_{\substack{F \subseteq \mathcal{E} \\ D(F) = \mathcal{S}}} \Pr(F) = \operatorname*{arg\,min}_{\substack{F \subseteq \mathcal{E} \\ D(F) = \mathcal{S}}} w(F). \tag{A.4}$$

Tesseract rewrites Eq. (A.4) as a shortest-path problem on the weighted directed acyclic graph

$$G := \big(2^{\mathcal{E}}, T, w\big), \tag{A.5}$$

whose vertices are all error subsets. A directed edge from F to $F' = F \cup \{e\}$ adds one previously absent error and has weight $w(e)$. More generally, the edge set is written

$$T := \big\{(F, F') : \ |F'| = |F| + 1, \ F \subset F', \ P(F, F') = 1\big\}, \tag{A.6}$$

where the predicate P controls which edges are retained. With $P \equiv 1$, every possible one-error extension is present. The start vertex and the valid exit vertices are

$$\text{START} := \varnothing, \qquad \text{EXIT} := \{F \subseteq \mathcal{E} : D(F) = \mathcal{S}\}. \tag{A.7}$$

Every path from START to a vertex F adds exactly the errors in F and therefore has total weight $w(F)$. Consequently, the minimum-cost path from START to any vertex in EXIT terminates at the MLE solution F^*.

A Dijkstra search would examine vertices in increasing accumulated cost. Tesseract accelerates this search with A^*: a state F is prioritized by $w(F) + h(F)$, where $h(F)$ estimates the remaining cost to an exit. For the exact canonical search, the residual detector set

$$x(F) := \mathcal{S} \oplus D(F) \tag{A.8}$$

is formed, and only errors incident to its lowest-index detector are considered. A precedence rule removes alternative orderings of the same additions while preserving one path to every candidate

error set. With an admissible heuristic and without approximate cutoffs, the first exit removed from the priority queue is therefore the exact solution of Eq. (A.4). The selected error subset, or equivalently its induced logical-observable or Pauli-frame action, is returned as the correction.

The paper also introduces practical accelerations—at-most-two pruning, beam cutoffs and beam climbing, priority-queue limits, detector-ordering ensembles, no-revisit residual detections, and detection penalties. Some of these cutoffs trade the exact MLE guarantee for lower runtime and memory use; their definitions and benchmark settings are given in Beni et al. [3].

A.2 Alternative Interpretation of Belief Propagation

A.2.1 Factor Graph

The Tanner graph is also a factor graph for the posterior distribution. Suppose that the variables have independent priors $P_j(e_j)$. Conditioning on a measured syndrome gives

$$P(e \mid s) = \frac{1}{Z(s)} \prod_{j=1}^{n} P_j(e_j) \prod_{i=1}^{m} \left[\bigoplus_{j \in \mathcal{N}(c_i)} e_j = s_i \right], \tag{A.9}$$

where $Z(s)$ is a normalization constant and the bracket is the indicator of the local parity constraint. Equivalently, each check node defines the factor

$$\psi_i(e_{\mathcal{N}(c_i)}) := \begin{cases} 1, & \bigoplus_{j \in \mathcal{N}(c_i)} e_j = s_i, \\ 0, & \text{otherwise.} \end{cases} \tag{A.10}$$

Variable nodes represent the unary prior factors, while check nodes represent the parity constraints. This factorization separates local evidence from the noise model and the global syndrome constraint into local terms, which is precisely the structure exploited by BP.

A.2.2 Bethe Free Energy

BP can be interpreted as a variational inference method that approximates the true posterior distribution by minimizing the Bethe free energy. Introducing approximate marginal distributions $b_i(e_i)$ and $b_a(e_{\mathcal{N}(a)})$, the Bethe free energy is given by:

$$F_{\text{Bethe}}(b) := \sum_{a} \sum_{e_{\mathcal{N}(a)}} b_a(e_{\mathcal{N}(a)}) \log \frac{b_a(e_{\mathcal{N}(a)})}{\psi_a(e_{\mathcal{N}(a)})} - \sum_{i} (d_i - 1) \sum_{e_i} b_i(e_i) \log b_i(e_i), \tag{A.11}$$

where ψ_a enforces the parity-check constraints and d_i denotes the degree of variable node i.

Under the Bethe–Peierls approximation, correlations between neighbouring regions are neglected, effectively assuming a locally tree-like structure. The fixed points of BP correspond to stationary points of the Bethe free energy.

However, quantum LDPC codes typically exhibit many short cycles, violating the assumptions underlying the Bethe approximation. As a result, BP may fail to converge or converge to suboptimal solutions.

A.2.3 Estimating Logical Class Masses with Belief Propagation

This appendix explains how BP can be used to approximate the probability mass associated with a logical class. The main idea is to express the logical class mass as the partition function of a constrained factor graph and then approximate this partition function using the Bethe free energy.

Logical class mass as a partition function

Let $e = (e_1, \ldots, e_n) \in \mathbb{F}_2^n$ denote a binary error vector, with independent prior $P(e) = \prod_{i=1}^n \phi_i(e_i)$, where $\phi_i(0) = 1 - p_i, \phi_i(1) = p_i$. Let H denote the parity-check matrix and L the logical observable matrix. Thus, the syndrome and logical label are $s = He$ and $\ell = Le$, respectively.

For a fixed syndrome s and logical label ℓ, define the *logical class mass*

$$Z_\ell(s) := \sum_{\substack{e:\, He=s \\ Le=\ell}} P(e). \tag{A.12}$$

It is the joint probability $Z_\ell(s) = P(S = s, \Lambda = \ell)$, and therefore

$$P(\Lambda = \ell \mid S = s) = \frac{Z_\ell(s)}{\sum_{\ell'} Z_{\ell'}(s)}. \tag{A.13}$$

To express Eq. (A.12) as a partition function, introduce the augmented constraint matrix

$$A := \begin{pmatrix} H \\ L \end{pmatrix}, \qquad y := \begin{pmatrix} s \\ \ell \end{pmatrix}. \tag{A.14}$$

For each row a of A, define a parity factor

$$\psi_a(e_{\mathcal{N}(a)}; y_a) := \mathbf{1}\left[\bigoplus_{i \in \mathcal{N}(a)} A_{ai} e_i = y_a\right]. \tag{A.15}$$

The corresponding factor-graph weight is

$$W_{s,\ell}(e) := \prod_i \phi_i(e_i) \prod_a \psi_a(e_{\mathcal{N}(a)}; y_a). \tag{A.16}$$

Its partition function is

$$Z_{[H;L]}(s, \ell) := \sum_e W_{s,\ell}(e) = \sum_{\substack{e:\, He=s \\ Le=\ell}} P(e) = Z_\ell(s). \tag{A.17}$$

Hence, estimating the partition function of the augmented factor graph is equivalent to estimating the probability mass of the corresponding logical class.

Partition-function estimate from BP

On a cycle-free factor graph, BP computes exact marginal quantities. In particular, a BP message can be interpreted as a partial partition function of the subtree on one side of an edge. Consequently, after BP has propagated through the entire tree, the global partition function can be recovered exactly.

For a loopy factor graph, the same BP update equations are applied, but the resulting partition function is generally approximate. At a BP fixed point, define the associated Bethe partition function by $Z_{\mathrm{B}} := \exp(-F_{\mathrm{B}})$, where F_{B} is the Bethe free energy.

A convenient expression for evaluating Z_{B} directly from the BP messages is

$$\log Z_{\mathrm{B}} = \sum_a \log Z_a + \sum_i \log Z_i - \sum_{(i,a)} \log Z_{ia} \tag{A.18}$$

with

$$Z_a := \sum_{e_{\mathcal{N}(a)}} \psi_a(e_{\mathcal{N}(a)}) \prod_{i \in \mathcal{N}(a)} m_{i \to a}(e_i), \quad Z_i := \sum_{e_i} \phi_i(e_i) \prod_{a \in \mathcal{N}(i)} m_{a \to i}(e_i), \quad Z_{ia} := \sum_{e_i} m_{i \to a}(e_i) m_{a \to i}(e_i). \tag{A.19}$$

The first two terms in Eq. (A.18) collect the local factor and variable contributions, while the edge

contribution is subtracted once to correct for double counting.

Applying Eq. (A.18) to the augmented $[H; L]$ factor graph with fixed (s, ℓ) gives the BP estimate

$$\widehat{Z}_\ell(s) \equiv Z_{\mathrm{B}}(s, \ell) \approx Z_\ell(s). \tag{A.20}$$

Therefore, the approximate logical posterior can be obtained as

$$\widehat{P}(\Lambda = \ell \mid S = s) := \frac{\widehat{Z}_\ell(s)}{\sum_{\ell'} \widehat{Z}_{\ell'}(s)}. \tag{A.21}$$

A decoder may then select $\hat{\ell} := \arg\max_\ell \widehat{Z}_\ell(s)$.

Thus, for each candidate logical label ℓ, one runs BP on the factor graph conditioned simultaneously on the observed syndrome s and the logical constraint $Le = \ell$, evaluates the local normalizers Z_a, Z_i, and Z_{ia}, and combines them through Eq. (A.18). The resulting value provides an estimate of the total probability mass of that logical class.

On trees this procedure is exact. For loopy Tanner graphs, $\widehat{Z}_\ell(s)$ is a Bethe approximation whose accuracy depends on the graph structure and on the BP fixed point. In particular, the standard Bethe interpretation assumes that BP has reached a fixed point; using messages from a non-converged trajectory should instead be regarded as a heuristic approximation.

A.3 Message-Dynamics Quantities

Table A.1 summarizes the quantities used in analyzing the message dynamics.

Group	Quantity	Definition	Interpretation
Message residual	Edge residual	$r_e^{(t)} = \left\lvert m_e^{(t+1)} - m_e^{(t)} \right\rvert$	One-step change of the check-to-variable message on e.
	Late residual	$R_e^{\text{late}} = \frac{1}{\lvert\mathcal{T}_{\text{late}}\rvert} \sum_{t \in \mathcal{T}_{\text{late}}} r_e^{(t)}$	Mean residual of edge e over the late iterations.
	Normalized late residual	$\widetilde{R}_e^{\text{late}} = \frac{R_e^{\text{late}}}{\lvert L_{i(e)}^{\text{prior}} \rvert + \epsilon}$	Normalizes out changes in the absolute prior-LLR scale, enabling comparisons across p.
	$R_{\max}$	$R_{\max} = \max_e \widetilde{R}_e^{\text{late}}$	Maximum normalized late residual over all edges.
	R_{mean}	$R_{\text{mean}} = \frac{1}{\lvert E \rvert} \sum_e \widetilde{R}_e^{\text{late}}$	Mean normalized late residual over all Tanner edges.
Sign-flip rate	Sign state	$\sigma_e^{(t)} = \begin{cases} +1, & m_e^{(t)} > \tau_0, \\ 0, & \lvert m_e^{(t)} \rvert \le \tau_0, \\ -1, & m_e^{(t)} < -\tau_0, \end{cases} \quad \tau_0 = 10^{-10}$	Discretized message preference with a zero dead zone.
	F_e	$F_e = \frac{\sum_t \mathbf{1}[\sigma_e^{(t+1)} \neq \sigma_e^{(t)}] \mathbf{1}[\sigma_e^{(t+1)} \neq 0] \mathbf{1}[\sigma_e^{(t)} \neq 0]}{N_e^{\text{comp}}}$	Edge-level fraction of comparable consecutive pairs exhibiting a genuine nonzero sign reversal.
	$F_{\max}$, F_{mean}	$F_{\max} = \max_e F_e, \qquad F_{\text{mean}} = \frac{1}{\lvert E \rvert} \sum_e F_e$	Sample-level maximum and mean sign-flip rates.
Period-2 strength	$D_{1,e}$	$D_{1,e} = \left\langle \left\lvert m_e^{(t+1)} - m_e^{(t)} \right\rvert \right\rangle_t$	Average one-step message movement.
	$D_{2,e}$	$D_{2,e} = \left\langle \left\lvert m_e^{(t+2)} - m_e^{(t)} \right\rvert \right\rangle_t$	Average two-step message movement.
	$P_e^{(2)}$	$P_e^{(2)} = \frac{D_{1,e}^2}{D_{1,e} + D_{2,e} + \epsilon}$	Large when one-step changes are strong but the message approximately returns after two iterations.
	$P_{\max}^{(2)}$	$P_{\max}^{(2)} = \max_e P_e^{(2)}$	Sample-level diagnostic of alternating dynamics. It is not used as a direct logical-failure score.
Variable-level belief instability	$b_i^{(t)}$	$b_i^{(t)} = \left\lvert L_i^{(t+1)} - L_i^{(t)} \right\rvert$	One-step movement of the posterior LLR of variable i.
	I_i^{belief}	$I_i^{\text{belief}} = \left\langle b_i^{(t)} \right\rangle_{t \in \mathcal{T}_{\text{late}}}$	Late-iteration posterior instability of variable i.
	$\overline{I}^{\text{belief}}$	$\overline{I}^{\text{belief}} = \frac{1}{n} \sum_i I_i^{\text{belief}}$	Sample-level mean belief-instability score.
Variable-level message instability	I_i^{msg}	$I_i^{\text{msg}} = \frac{1}{\lvert\mathcal{N}(i)\rvert} \sum_{a \in \mathcal{N}(i)} R_{a \to i}^{\text{late}}$	Mean late residual over the incoming check-to-variable edges of variable i. Unlike $R_{\max}$, this quantity is not normalized by the prior LLR.
	$\overline{I}^{\text{msg}}$	$\overline{I}^{\text{msg}} = \frac{1}{n} \sum_i I_i^{\text{msg}}$	Sample-level mean message-instability score.
Extrinsic-conflict score	$C_i^{(t)}$	$C_i^{(t)} = 1 - \frac{\left\lvert \sum_{a \in \mathcal{N}(i)} m_{a \to i}^{(t)} \right\rvert}{\sum_{a \in \mathcal{N}(i)} \left\lvert m_{a \to i}^{(t)} \right\rvert + \epsilon}$	$C_i^{(t)} \approx 0$ when incoming messages are aligned, whereas $C_i^{(t)} \approx 1$ when large incoming messages largely cancel.
	C_i^{late}	$C_i^{\text{late}} = \left\langle C_i^{(t)} \right\rangle_{t \in \mathcal{T}_{\text{late}}}$	Late-iteration extrinsic-conflict score of variable i.
	$\overline{C}$	$\overline{C} = \frac{1}{n} \sum_i C_i^{\text{late}}$	Sample-level average extrinsic-conflict score.
Operational stability	Stability criterion	$R_{\max} < \tau$	A trajectory satisfying the threshold is classified as operationally stable.

Table A.1: Summary of the quantities used to characterize message dynamics. Here, $e = (c \to v)$ denotes a directed check-to-variable edge, $\mathcal{T}_{\text{late}}$ the set of late iterations, and ϵ a regularization constant.

A.4 OSD Reliability Metrics

Table A.2 summarizes the reliability metrics tested when using BP soft information as input to OSD.

Method	Formula	Interpretation
`final_abs`	$\lvert L(T)\rvert$	Baseline using only the LLR magnitude from the final BP iteration.
`temporal_mean`	$\lvert\mathrm{mean}(L(T-w:T))\rvert$	Averages the LLRs over the last w iterations; sign changes and oscillations reduce the score.
`temporal_median`	$\lvert\mathrm{median}(L(T-w:T))\rvert$	Takes the median of the LLRs over the last w iterations, reducing the influence of anomalous iterations.
`instability`	$\dfrac{\lvert L(T)\rvert}{1+\beta I}$	Penalizes the final LLR using dynamic instability. I is selected from message instability, belief instability, sign flip, and extrinsic conflict based on the best performance.
`sign_consistency`	$\lvert L(T)\rvert\,\lvert\mathrm{mean}(\mathrm{sign}(L))\rvert^2$	Uses the LLRs from the last few iterations; the more stable the sign, the closer the score is to the final LLR magnitude.
`snr`	$\dfrac{\lvert\mathrm{mean}(L)\rvert}{\mathrm{std}(L)+\epsilon}$	Uses the LLRs from the last few iterations; the score is high when the mean magnitude is large and the temporal fluctuation is small.
`conflict`	$\dfrac{\lvert L(T)\rvert}{1+\beta C}$	Directly penalizes conflicting messages from neighbouring check nodes.
`combined`	$\mathrm{rank}(\lvert L(T)\rvert) - 0.5\,\mathrm{rank}(\texttt{extrinsic_conflict}) - \mathrm{rank}(\texttt{rank_volatility})$	Combines the final LLR magnitude, extrinsic conflict, and reliability-rank volatility.

Table A.2: Reliability metrics tested for constructing OSD reliability orderings from BP trajectories.

A.4.1 Online OSD Handoff and Prefix-buffer Selection

Let $L^{(t)} := \left(L_1^{(t)}, \ldots, L_n^{(t)}\right)$ denote the posterior BP reliability after iteration t. The trajectory experiments show that some useful reliability ordering may appear at an intermediate iteration and disappear before the endpoint. We therefore consider two strategies for selecting which BP snapshot is passed to OSD-1.

For each iteration, a snapshot-quality model estimates

$$\hat{p}_1(t) := P\Big(\text{OSD-1}(L^{(t)}) \text{ is logically correct} \mid x_{\le t}\Big), \tag{A.22}$$

where $x_{\le t}$ contains only information available up to iteration t, including the current reliability and optionally its recent message-dynamics.

Online handoff. The online-handoff policy stops BP as soon as the predicted OSD-1 success probability exceeds a threshold τ:

$$t_{\text{handoff}} := \min\left\{t : \hat{p}_1(t) \ge \tau\right\}. \tag{A.23}$$

OSD-1 is then applied immediately using $L^{(t_{\text{handoff}})}$. Thus, the policy attempts to exploit a favorable reliability window as soon as it is detected, rather than waiting for the conventional BP endpoint.

Prefix-buffer selection. The prefix-buffer policy does not commit to a snapshot immediately. Instead, it stores the BP reliabilities already observed before the standard endpoint t_{end}: $L^{(1)}, L^{(2)}, \ldots, L^{(t_{\text{end}})}$.

At t_{end}, it selects the stored snapshot with the largest predicted OSD-1 success probability,

$$t^\star := \arg \max_{1 \le t \le t_{\text{end}}} \hat{p}_1(t), \tag{A.24}$$

and applies OSD-1($L^{(t^\star)}$). This allows the decoder to recover an earlier transiently useful reliability ordering even if it was not recognized strongly enough to trigger an immediate handoff when it first appeared.

Both strategies use only BP snapshots that have already been observed. Their difference is that online handoff makes an immediate decision when the threshold is first crossed, whereas prefix buffering defers the decision and selects the highest-scoring snapshot from the observed reliabilities. Neither method changes the OSD algorithm itself; they modify only which BP reliability snapshot is supplied to OSD.

A.5 Adaptive BP

Adaptive BP constructs several alternative Tanner graph representations from the trajectory of a single BP run on the original parity-check matrix. Its purpose is to perturb the local Tanner graph structure around variables whose beliefs are unstable, while leaving the underlying decoding constraints and priors unchanged.

Given a syndrome s, BP is first run on the original parity-check matrix H while recording the posterior LLR trajectory. Each variable v is assigned an instability score I_v computed from the trajectory. The k highest-scoring variables form the unstable set $\mathcal{U}$.

Candidate redundant checks are then generated locally around $\mathcal{U}$. Let $\mathcal{N}(\mathcal{U})$ denote the original parity checks adjacent to at least one unstable variable. For pairs of checks $a, b \in \mathcal{N}(\mathcal{U})$, a redundant row is constructed as

$$r_{ab} := H_a \oplus H_b, \qquad s_{ab} := s_a \oplus s_b. \tag{A.25}$$

Only non-trivial and support-distinct rows that intersect the unstable set are retained. These candidates are greedily grouped into a small number of augmented parity-check matrix views. The construction favors rows that cover unstable variables not yet represented in the current view while avoiding unnecessarily large redundant checks.

For each view j, the selected redundant rows $R^{(j)}$ are appended to the original parity-check matrix,

$$H^{(j)} := \begin{bmatrix} H \\ R^{(j)} \end{bmatrix}, \qquad \mathbf{s}^{(j)} := \begin{bmatrix} \mathbf{s} \\ \mathbf{s}_R^{(j)} \end{bmatrix}. \tag{A.26}$$

Each view is constructed independently from the original H rather than stacked cumulatively, and BP is run once on every prepared view with unchanged priors.

The candidate set consists of the main-BP hard decision and hard decisions from auxiliary views. Every candidate is validated against the original syndrome equation $H\hat{e} = s$. Among the syndrome-consistent candidates, Adaptive BP returns the one with minimum negative log-likelihood under the original priors.

$$\text{NLL}(\hat{e}) := -\sum_v \left[\hat{e}_v \log p_v + (1 - \hat{e}_v) \log(1 - p_v)\right]. \tag{A.27}$$

If no candidate satisfies the original syndrome, the main-BP output is returned by default.

Bibliography

[1] Abdelaziz Amraoui et al. "Finite-Length Scaling for Iteratively Decoded LDPC Ensembles". In: *IEEE Transactions on Information Theory* 55.2 (2009), pp. 473–498. DOI: 10.1109/TIT.2008.2009580.

[2] Michele Banfi et al. *Best-First Ordered Statistics Decoding of Quantum LDPC Codes*. 2026. arXiv: 2605.25777 [cs.IT]. URL: https://arxiv.org/abs/2605.25777.

[3] Laleh Aghababaie Beni, Oscar Higgott, and Noah Shutty. *Tesseract: A Search-Based Decoder for Quantum Error Correction*. 2025. arXiv: 2503.10988 [quant-ph]. URL: https://arxiv.org/abs/2503.10988.

[4] Anthony Benois et al. *Optimal Decoding of Small Codes by Density Matrix Propagation*. 2026. arXiv: 2606.14455 [quant-ph]. URL: https://arxiv.org/abs/2606.14455.

[5] Sergey Bravyi, David Poulin, and Barbara Terhal. "Tradeoffs for Reliable Quantum Information Storage in 2D Systems". In: *Phys. Rev. Lett.* 104 (5 Feb. 2010), p. 050503. DOI: 10.1103/PhysRevLett.104.050503. URL: https://link.aps.org/doi/10.1103/PhysRevLett.104.050503.

[6] Sergey Bravyi et al. "High-threshold and low-overhead fault-tolerant quantum memory". en. In: *Nature* 627.8005 (2024), pp. 778–782.

[7] A. R. Calderbank and Peter W. Shor. "Good quantum error-correcting codes exist". In: *Phys. Rev. A* 54 (2 Aug. 1996), pp. 1098–1105. DOI: 10.1103/PhysRevA.54.1098. URL: https://link.aps.org/doi/10.1103/PhysRevA.54.1098.

[8] Eric Dennis et al. "Topological quantum memory". In: *Journal of Mathematical Physics* 43.9 (2002), pp. 4452–4505. DOI: 10.1063/1.1499754.

[9] Peter-Jan H. S. Derks et al. "Designing Fault-Tolerant Circuits Using Detector Error Models". In: *Quantum* 9 (2025), p. 1905. DOI: 10.22331/q-2025-11-06-1905. arXiv: 2407.13826 [quant-ph].

[10] Diego Forlivesi, Lorenzo Valentini, and Marco Chiani. "Performance Analysis of Quantum CSS Error-Correcting Codes via MacWilliams Identities". In: *Quantum* 9 (2025), p. 1950. DOI: 10.22331/q-2025-12-16-1950.

[11] R. Gallager. "Low-density parity-check codes". In: *IRE Transactions on Information Theory* 8.1 (1962), pp. 21–28. DOI: 10.1109/TIT.1962.1057683.

[12] Craig Gidney. "Stim: a fast stabilizer circuit simulator". In: *Quantum* 5 (July 2021), p. 497. ISSN: 2521-327X. DOI: 10.22331/q-2021-07-06-497. URL: https://doi.org/10.22331/q-2021-07-06-497.

[13] Daniel Gottesman. *Stabilizer Codes and Quantum Error Correction*. 1997. arXiv: quant-ph/9705052 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9705052.

[14] Antonio deMarti iOlius et al. "An almost-linear time decoding algorithm for quantum LDPC codes under circuit-level noise". en. In: *NPJ Quantum Inf.* (2026).

[15] Antonio deMarti iOlius et al. "Decoding algorithms for surface codes". In: *Quantum* 8 (Oct. 2024), p. 1498. ISSN: 2521-327X. DOI: 10.22331/q-2024-10-10-1498. URL: http://dx.doi.org/10.22331/q-2024-10-10-1498.

[16] Pavithran Iyer and David Poulin. *Hardness of decoding quantum stabilizer codes*. 2013. arXiv: 1310.3235 [quant-ph]. URL: https://arxiv.org/abs/1310.3235.

[17] Kenta Kasai. *Bringing Classical LDPC Design Principles to Quantum LDPC Codes*. 2026. URL: https://kasai.ict.eng.isct.ac.jp/files/Boston2026.pdf.

[18] Emanuel Knill, Raymond Laflamme, and Lorenza Viola. "Theory of Quantum Error Correction for General Noise". In: *Phys. Rev. Lett.* 84 (11 Mar. 2000), pp. 2525–2528. DOI: 10.1103/PhysRevLett.84.2525. URL: https://link.aps.org/doi/10.1103/PhysRevLett.84.2525.

[19] Ching-Feng Kung, Kao-Yueh Kuo, and Ching-Yi Lai. *Efficient Approximate Degenerate Ordered Statistics Decoding for Quantum Codes via Reliable Subset Reduction*. 2026. arXiv: 2412.21118 [quant-ph]. URL: https://arxiv.org/abs/2412.21118.

[20] Arshpreet Singh Maan et al. "Decoding correlated errors in quantum LDPC codes". en. In: *Nat. Commun.* 17.1 (2026).

[21] M. E. J. Newman and Cristopher Moore. "Glassy dynamics and aging in an exactly solvable spin model". In: *Phys. Rev. E* 60 (5 Nov. 1999), pp. 5068–5072. DOI: 10.1103/PhysRevE.60.5068. URL: https://link.aps.org/doi/10.1103/PhysRevE.60.5068.

[22] OEIS Foundation Inc. *A001316: Gould's sequence*. The On-Line Encyclopedia of Integer Sequences. Sequence A001316.

[23] Pavel Panteleev and Gleb Kalachev. "Degenerate quantum LDPC codes with good finite length performance". en. In: *Quantum* 5 (2021), p. 585. DOI: 10.22331/q-2021-11-22-585.

[24] David Poulin and Yeojin Chung. "On the Iterative Decoding of Sparse Quantum Codes". In: *Quantum Information and Computation* 8.10 (2008), pp. 987–1000. DOI: 10.26421/QIC8.10-8.

[25] Sheida Rabeti and Hessam Mahdavifar. *Multiple-Bases Belief Propagation List Decoding for Quantum LDPC Codes*. 2026. arXiv: 2605.14170 [cs.IT]. URL: https://arxiv.org/abs/2605.14170.

[26] Nithin Raveendran and Bane Vasić. "Trapping sets of quantum LDPC codes". en. In: *Quantum* 5.562 (2021), p. 562.

[27] Shaked Regev, Daniel Dilley, and Ryan Bennink. *Closed form logical error rate approximations for surface codes*. 2026. arXiv: 2605.03054 [quant-ph].

[28] Tom Richardson. "Error Floors of LDPC Codes". In: *Proceedings of the 41st Annual Allerton Conference on Communication, Control, and Computing*. 2003.

[29] L. Riesebos et al. "Pauli frames for quantum computer architectures". In: *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*. 2017, pp. 1–6. DOI: 10.1145/3061639.3062300.

[30] Joschka Roffe et al. "Decoding across the quantum low-density parity-check code landscape". In: *Phys. Rev. Res.* 2 (4 Dec. 2020), p. 043423. DOI: 10.1103/PhysRevResearch.2.043423. URL: https://link.aps.org/doi/10.1103/PhysRevResearch.2.043423.

[31] Diego Ruiz et al. "LDPC-cat codes for low-overhead quantum computing in 2D". In: *Nature Communications* 16.1 (Jan. 2025). ISSN: 2041-1723. DOI: 10.1038/s41467-025-56298-8. URL: http://dx.doi.org/10.1038/s41467-025-56298-8.

[32] R. Tanner. "A recursive approach to low complexity codes". In: *IEEE Transactions on Information Theory* 27.5 (1981), pp. 533–547. DOI: 10.1109/TIT.1981.1056404.

[33] Jean-Pierre Tillich and Gilles Zémor. "Quantum LDPC Codes With Positive Rate and Minimum Distance Proportional to the Square Root of the Blocklength". In: *IEEE Transactions on Information Theory* 60.2 (2014), pp. 1193–1202. DOI: 10.1109/TIT.2013.2292061.

[34] Joel J. Wallman and Joseph Emerson. "Noise tailoring for scalable quantum computation via randomized compiling". In: *Phys. Rev. A* 94 (5 Nov. 2016), p. 052325. DOI: 10.1103/PhysRevA.94.052325. URL: https://link.aps.org/doi/10.1103/PhysRevA.94.052325.

[35] Hanwen Yao et al. "Belief Propagation Decoding of Quantum LDPC Codes with Guided Decimation". In: *2024 IEEE International Symposium on Information Theory (ISIT)*. 2024, pp. 2478–2483. DOI: 10.1109/ISIT57864.2024.10619083.

[36] Theodore J. Yoder et al. *Tour de gross: A modular quantum computer based on bivariate bicycle codes*. 2025. arXiv: 2506.03094 [quant-ph]. URL: https://arxiv.org/abs/2506.03094.